



UNIVERSIDAD DE
COSTA RICA

ANALISIS NUMERICO

MA0501

TAREA 1

Estudiantes:

María Paula Jiménez Torres - C04095
Andy Roberto Peralta Duarte - C25827
Dominick Rodríguez Trejos - B76600
Holmar Rivera Castellon - B86564

Profesor:

Esteban Segura Ugalde

II Semestre 2026

Índice

1. Ejercicio 1	2
2. Ejercicio 2	4
3. Ejercicio 3	6
4. Ejercicio 4	7
5. Ejercicio 5	10
6. Ejercicio 6	13
7. Ejercicio 7	17
8. Ejercicio 8	20
9. Ejercicio 9	22
10. Ejercicio 10	27
11. Ejercicio 11	30
12. Ejercicio 12	33
13. Ejercicio 13	37
14. Ejercicio 14	40
15. Ejercicio 15	42

1. Ejercicio 1

Enunciado: El método de división en múltiples secciones es la versión generalizada de Bisección. Este metodo busca aproximar las raíces de una función continua $f(x) = 0$ dividiendo el intervalo $[a, b]$ en n subintervalos de igual longitud $h = \frac{b-a}{n}$ en cada iteración. Para llevar a cabo el método, se siguen los siguientes pasos:

- Verificar que la función f cumple el Teorema de Bolzano en el intervalo $[a, b]$ ($f(a) \cdot f(b) \leq 0$).
- Subdividir el intervalo actual $[a_k, b_k]$ en n subintervalos mediante los puntos $s_i = a_k + i \cdot h$ para $i = 0, 1, \dots, n$.
- Identificar el subintervalo $[s_{i-1}, s_i]$ donde se encuentra la raíz mediante el cambio de signo $f(s_{i-1}) \cdot f(s_i) < 0$.
- Actualizar el intervalo de búsqueda definiendo $[a_{k+1}, b_{k+1}] := [s_{i-1}, s_i]$.
- Aproximar la raíz de f mediante el punto medio del nuevo intervalo: $x_k = \frac{a_{k+1} + b_{k+1}}{2}$.
- Calcular el error $e_k = |a_{k+1} - b_{k+1}|$ y verificar si se cumple el criterio de parada $e_k < \text{tol}$.

A continuación se muestra la implementación de la función en MATLAB:

Listing 1: Implementación de la función `division_multiples_secciones`

```
1 function [M, cont] = division_multiples_secciones(f, a, b, n, tol)
2     % division_multiples_secciones busca una raíz de f dividiendo el
   intervalo [a, b] en 'n' subintervalos por iteracion. Es la
   version general de Biseccion.
3
4     % Entradas:
5     % f      - funcion
6     % [a,b]  - intervalo de busqueda de la solucion
7     % n      - numero de subintervalos
8     % tol    - tolerancia para detener el programa
9
10    % Salidas:
11    % M      - Matriz de resultados
12    % cont   - Numero total de iteraciones
13
14    cont = 0;
15    er = abs(b - a);
16
17    %Verificacion inicial de existencia de raiz
18    if (f(a) * f(b) > 0)
19        warning('El metodo puede no funcionar ya que no se cumplen las
   condiciones del teorema de Bolzano.');
```

```
20    M = [];
21    return;
22 end
```

```

23
24 while er > tol
25     h = (b - a) / n;
26     a_nuevo = a;
27     b_nuevo = b;
28
29     % Recorre los n subintervalos para hallar en cual esta la raiz
30     for i = 1:n
31
32         s_i_prev = a + (i - 1) * h;
33         s_i       = a + i * h;
34
35         % verifica si hay un cero exacto en el extremo del
36         subintervalo
37         if f(s_i) == 0
38             a_nuevo = s_i;
39             b_nuevo = s_i;
40             break;
41         end
42
43         % Busqueda del cambio de signo: f(s_{i-1}) * f(s_i) < 0
44         if f(s_i_prev) * f(s_i) < 0
45             a_nuevo = s_i_prev;
46             b_nuevo = s_i;
47             break;
48         end
49     end
50
51     % Actualizar a y b con el nuevo intervalo hallado
52     a = a_nuevo;
53     b = b_nuevo;
54
55     x_k = (a + b) / 2;
56     er = abs(b - a);
57     e_k = |a_{k+1} - b_{k+1}|;
58
59     cont = cont + 1;
60     M(cont, 1) = x_k;
61     M(cont, 2) = er;
62     M(cont, 3) = a;
63     M(cont, 4) = b;
64
65     if a == b
66         break;
67     end
68 end

```

Para probar la implementación se utilizó la función $f(x) = e^x - 2x - 1$ en el intervalo $[1, 2]$, el cual es un ejemplo visto en clase, con $n = 4$ y una tolerancia de $\text{tol} = 10^{-6}$. La siguiente tabla sintetiza los resultados obtenidos en cada iteración k :

Cuadro 1: Matriz M de iteraciones para $f(x) = e^x - 2x - 1$ con $n = 4$ y $\text{tol} = 10^{-6}$

Iteración (k)	Aproximación (x_k)	Error (e_k)	Límite inf. (a_k)	Límite sup. (b_k)
1	1,3750000	$2,500000 \times 10^{-1}$	1,2500000	1,5000000
2	1,2812500	$6,250000 \times 10^{-2}$	1,2500000	1,3125000
3	1,2578125	$1,562500 \times 10^{-2}$	1,2500000	1,2656250
4	1,2558594	$3,906250 \times 10^{-3}$	1,2539063	1,2578125
5	1,2563477	$9,765625 \times 10^{-4}$	1,2558594	1,2568359
6	1,2564697	$2,441406 \times 10^{-4}$	1,2563477	1,2565918
7	1,2564392	$6,103516 \times 10^{-5}$	1,2564087	1,2564697
8	1,2564316	$1,525879 \times 10^{-5}$	1,2564240	1,2564392
9	1,2564297	$3,814697 \times 10^{-6}$	1,2564278	1,2564316
10	1,2564311	$9,536743 \times 10^{-7}$	1,2564306	1,2564316

El algoritmo convergió en 10 iteraciones, obteniendo como aproximación final $x_{10} \approx 1,2564311$ con un error final $e_{10} \approx 9,5367 \times 10^{-7} < 10^{-6}$.

2. Ejercicio 2

Enunciado: La idea de este ejercicio es programar el método iterativo de gradiente conjugado con preconditionador para aproximar la solución de $Ax = b$. En este caso, $A \in \mathbb{R}^{nn}$ debe ser una matriz simétrica ($A^t = A$) y definida positiva (todos sus valores propios son positivos), y $b \in \mathbb{R}^n$

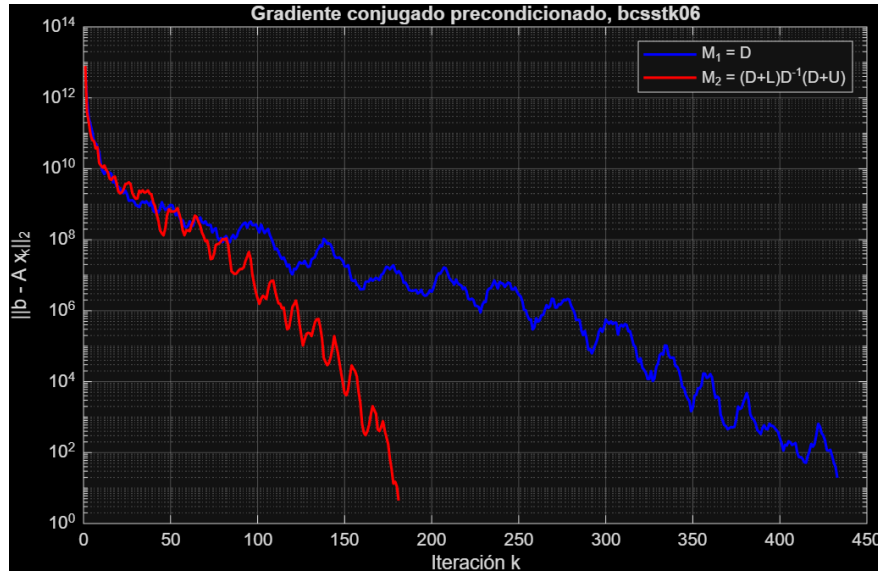
El algoritmo se implementa desde MATLAB siguiendo las instrucciones y descripción del algoritmo dado en el enunciado. Para probar el método se utiliza la matriz `bcsstk06.mat`, la cuál es una matriz de 420 filas y 420 columnas.

(e) Comparación de los errores absolutos de los dos casos y del método Gauss-Seidel

A partir de la matriz `bcsstk06.mat` se obtienen la dos matriz preconditionadoras M_1 y M_2 , así como el vector de constantes b , se prueba el algoritmo con una aproximación inicial $x_0 = 0$ y una tolerancia de convergencia de 10^{-6} .

Cuando se usa la matriz M_1 se obtiene una aproximación después de 433 iteraciones, mientras que al usar M_2 la aproximación se obtiene después de 181 ciclos, se guardan los resultados para el error absoluto $\|b - Ax_k\|$ y en la siguiente figura se pueden ver estos resultados obtenidos:

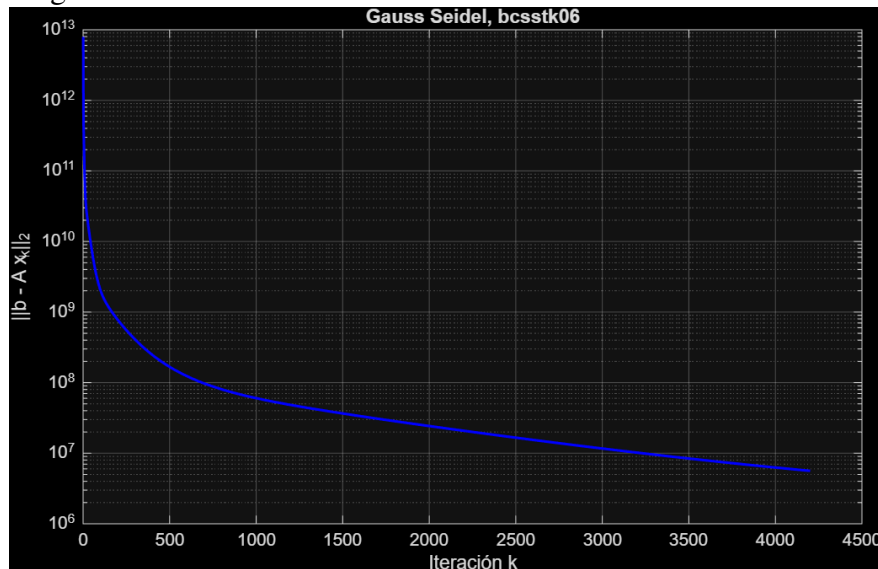
Figura 1: Evolución del error absoluto del método de gradiente conjugado con preconditionadores M_1 y M_2



Se puede ver que el error al usar M_2 disminuye más rápido que al usar M_1 , por esto produce aproximaciones con un error absoluto menor en menos iteraciones. Cabe destacar que en ambos casos el error disminuye en 12 ordenes de magnitud, pero M_2 es preferible pues logra esta caída en menos iteraciones. Cabe destacar que ninguno de los casos alcanza un error absoluto menor a la tolerancia establecida, pero esto se debe a que el criterio de parada del algoritmo se establece como $\|\alpha_k p_k\| \leq 10^{-6}$ en vez de comparar al error de cada iteración con la tolerancia.

Además, se implemento el método de Gauss-Seidel realizando algunos cambios al método de Jacobi trabajado en clases y se probó con la misma matriz `bcsstk06.mat`, a continuación se puede ver la evolución del error absoluto para el método de Gauss-Seidel:

Figura 2: Evolución del error absoluto del método de Gauss-Seidel



Note que en comparación a los otros dos casos, Gauss-Seidel no logra la misma caída de 12 ordenes de magnitud, incluso después de la máxima cantidad de iteraciones permitidas de 4.200, el método converge demasiado lento en comparación a los otros dos, e incluso requiere aumentar exageradamente la cantidad máxima de iteraciones permitidas.

3. Ejercicio 3

Enunciado: Considere la ecuación:

$$2(\alpha + 2)x + \alpha \sin(2x) = 0$$

(a) Empleando el Teorema de Valores Intermedios, determine todos los valores de α para los cuales la ecuación tiene solución en $[1, 2]$.

Sea $f(x) = 2(\alpha + 2)x + \alpha \sin(2x)$, f es continua en $[1, 2]$, según el teorema de valores intermedios para que exista un $x \in [1, 2]$ tal que $f(x) = 0$ debe suceder que $f(1) \leq 0 \leq f(2)$, así

$$\begin{aligned} f(1) &= 2\alpha + 4 + \alpha \sin(2) \leq 0 \\ \implies \alpha &\leq \frac{-4}{2 + \sin(2)} \approx -1,374902 \end{aligned}$$

Luego

$$\begin{aligned} f(2) &= 4\alpha + 8 + \alpha \sin(4) \geq 0 \\ \implies \alpha &\geq \frac{-8}{4 + \sin(4)} \approx -2,455701 \\ \implies \alpha &\in \left[\frac{-8}{4 + \sin(4)}, \frac{-4}{2 + \sin(2)} \right] \end{aligned}$$

(b) Muestre que la ecuación se puede escribir como un problema de hallar un punto fijo de $x = \varphi(x)$, donde

$$\varphi(x) := -\frac{\alpha x}{2} - \frac{\alpha \sin(2x)}{4}$$

Además, determine todos los valores de α para los cuales la función φ admite al menos un punto fijo en $[1, 2]$

$$\begin{aligned} 2(\alpha + 2)x + \alpha \sin(2x) &= 0 \\ \alpha x + 2x &= \frac{-\alpha \sin(2x)}{2} \\ x &= -\frac{\alpha x}{2} - \frac{\alpha \sin(2x)}{4} \end{aligned}$$

Existe un punto fijo si para todo $x \in [1, 2]$ se cumple que $\varphi(x) \in [1, 2]$. Note que

$$\varphi(1) = -\frac{\alpha}{2} - \frac{\alpha \sin(2)}{4} \geq 1$$

$$\Rightarrow \alpha \leq \frac{-4}{2 + \sin(2)}$$

$$\varphi(2) = -\alpha - \frac{\alpha \sin(4)}{4} \leq 2$$

$$\Rightarrow \alpha \geq \frac{-8}{4 + \sin(4)}$$

Además

$$\varphi'(x) = -\frac{\alpha}{2} - \frac{\alpha}{2} \cos(2x) \geq 0$$

Para todo $x \in [1, 2]$ con $\alpha < 0$

$$\Rightarrow 1 \leq \varphi(1) \leq \varphi(2) \leq 2$$

Existe un punto fijo si α

(c) Con base en los resultados obtenidos en las partes a) y b), justifique si es mejor usar el método de bisección en la ecuación o el método de punto fijo en la ecuación $x = \varphi(x)$.

El método de punto fijo es más provechoso pues la raíz podría estar muy cerca de 1 ó 2 según α lo cual vuelve el método de bisección muy ineficiente.

(d) Aplique el método de punto fijo y determine una aproximación para la solución de la ecuación, con una tolerancia de 10^{-4} empleando el error relativo en cada iteración. Use $x_0 = 1$ y $\alpha = -2$.

Por punto fijo

$$x_{k+1} = \varphi(x_k)$$

Si $x_0 = 1$ y $\alpha = -2$

$$x_1 = 1 + \frac{\sin(2)}{2} = 1,454649$$

con error relativo = 0,454649. Así

$$x_2 = 1,569755 \quad \text{err} = 0,073327$$

$$x_3 = 1,570796 \quad \text{err} = 0,000667$$

$$x_4 = 1,570796 \quad \text{err} = 0,000000$$

4. Ejercicio 4

Enunciado: Considere la sucesión definida por:

$$x_{k+1} = \frac{20x_k + \frac{21}{x_k^2}}{21}, \text{ para } k \geq 1$$

(a) Muestre que el límite cuando $k \rightarrow \infty$ de la sucesión es $\sqrt[3]{21}$

Considere la función f , donde

$$f(x) = \frac{20x + \frac{21}{x^2}}{21}$$

Entonces $x_{k+1} = f(x_k)$. Note que f es continua en el intervalo $[2, 3]$ pues no existen indeterminaciones en él. Luego

$$f'(x) = \frac{20}{21} - \frac{2}{x^3} > 0$$

para todo x en $[2, 3]$, es decir $f(x)$ es creciente en el intervalo así

$$f(2) = \frac{181}{84} \approx 2,154761 \in [2, 3]$$

$$f(3) = \frac{187}{63} \approx 2,968253 \in [2, 3]$$

entonces $f(x) \in [2, 3]$ para todo x en $[2, 3]$. Además

$$|f'(x)| < 0,9$$

Pues $f'(3) = \frac{166}{189} \approx 0,878306$, el máximo dentro del intervalo. Por lo tanto la sucesión x_{k+1} converge a un punto fijo dentro de $[2, 3]$. Como x_{k+1} converge, existe L tal que

$$\begin{aligned} \lim_{k \rightarrow \infty} x_{k+1} &= L \\ \Rightarrow \lim_{k \rightarrow \infty} \frac{20x_k + \frac{21}{x_k^2}}{21} &= L \\ \Rightarrow \frac{20L + \frac{21}{L^2}}{21} &= L \\ \frac{20L^3 + 21}{L^2} &= 21L \\ 21 &= L^3 \\ L &= \sqrt[3]{21} \end{aligned}$$

(b) Determine el orden de convergencia de la sucesión. Además, explique si la convergencia es cuadrática.

f es continuamente diferenciable, además

$$f(\sqrt[3]{21}) = \sqrt[3]{21}$$

y

$$|f'(\sqrt[3]{21})| = \frac{6}{7} < 1$$

Por lo tanto la sucesión converge linealmente.

(c) Realice 4 iteraciones de la sucesión para aproximar $\sqrt[3]{21}$ con $x_0 = 1$. Además, calcule el error absoluto en cada iteración.

$x_0 = 1$	
$x_1 = \frac{41}{21}$	err = 0,95238095
$x_2 = 2,12175427$	err = 0,16937332
$x_3 = 2,24284969$	err = 0,12109542
$x_4 = 2,33483967$	err = 0,09198998

(d) Calcule en su calculadora o computadora $\sqrt[3]{21}$ (esto con redondeo a 8 dígitos después de la coma). Una vez obtenido este resultado: calcule el número de dígitos significativos del resultado obtenido en el punto anterior.

calculadora: $\sqrt[3]{21} = 2,75892418$. Luego

$$\frac{|2,75892418 - 2,33483967|}{|2,75892418|} = 0,1537137$$

$$0,1537137 < 5 \times 10^{-1} = 0,5$$

Se obtuvo solo 1 dígito significativo con 4 iteraciones.

(e)

```

1 function resultado = aproximarRaiz(x0, tolerancia)
2 % Funcion para aproximar la raiz cubica de veintiuno con una sucesion
3 % especifica
4 % Entradas:      x0 --- estimacion inicial
5 %               tolerancia --- precision deseada por el usuario
6 %
7 % Salidas:      resultado --- la estimacion del valor real
8
9 if (x0 > 0 && tolerancia > 0)
10     contador = 1;
11     er = 1;
12     while (contador < 200 && er > tolerancia)
13         resultado = (20*x0 + 21/x0^2)/21;
14         er = abs(resultado - x0);
15         x0 = resultado;

```

```

16     contador = contador + 1;
17     end
18 else
19     error("el valor inicial y la tolerancia deben ser positivos")
20 end

```

5. Ejercicio 5

Enunciado: Sea $\mathbf{A}$ una matriz 4×4 a la cual se le realizaron las siguientes operaciones elementales entre filas:

$$f_2 - \frac{1}{2}f_1, \quad f_3 - 3f_1, \quad f_4 + 4f_1, \quad f_3 + \frac{1}{2}f_2 \quad \text{y} \quad f_4 - f_2,$$

obteniendo como resultado la matriz:

$$\mathbf{U} = \begin{pmatrix} 7 & -1 & 2 & 9 \\ 0 & -1 & -3 & 0 \\ 0 & 0 & 5 & 2 \\ 0 & 0 & 10 & 4 \end{pmatrix}$$

(a) Determinar explícitamente la matriz $\mathbf{A}$

Solución: Las operaciones elementales dadas representan el proceso de eliminación gaussiana sin pivoteo. Cada operación se asocia a una matriz de Gauss elemental E_k , de manera que la matriz resultante es $\mathbf{U} = E_5 E_4 E_3 E_2 E_1 \mathbf{A}$.

Como la factorización de Gauss es $\mathbf{A} = \mathbf{L}\mathbf{U}$, la matriz $\mathbf{L}$ se obtiene al revertir el efecto de las operaciones elementales de fila en el orden correspondiente. Los multiplicadores $\ell_{i,j}$ para $f_i - \ell_{i,j}f_j$ se colocan directamente en la entrada (i, j) de $\mathbf{L}$:

1. De $f_2 - \frac{1}{2}f_1 \implies \ell_{2,1} = \frac{1}{2}$
2. De $f_3 - 3f_1 \implies \ell_{3,1} = 3$
3. De $f_4 + 4f_1 = f_4 - (-4)f_1 \implies \ell_{4,1} = -4$
4. De $f_3 + \frac{1}{2}f_2 = f_3 - (-\frac{1}{2})f_2 \implies \ell_{3,2} = -\frac{1}{2}$
5. De $f_4 - f_2 \implies \ell_{4,2} = 1$
6. No se realizó ninguna operación de la fila 3 sobre la fila 4, por lo que $\ell_{4,3} = 0$.

Por lo tanto, la matriz triangular inferior unitaria $\mathbf{L}$ es:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 \\ 3 & -\frac{1}{2} & 1 & 0 \\ -4 & 1 & 0 & 1 \end{pmatrix}$$

Para determinar la matriz $\mathbf{A}$, calculamos el producto $\mathbf{A} = \mathbf{L}\mathbf{U}$:

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 \\ 3 & -\frac{1}{2} & 1 & 0 \\ -4 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 7 & -1 & 2 & 9 \\ 0 & -1 & -3 & 0 \\ 0 & 0 & 5 & 2 \\ 0 & 0 & 10 & 4 \end{pmatrix}$$

Realizando el cálculo fila por fila:

$$\text{Fila 1: } (7, -1, 2, 9)$$

$$\text{Fila 2: } \frac{1}{2}(7, -1, 2, 9) + 1(0, -1, -3, 0) = \left(\frac{7}{2}, -\frac{3}{2}, -2, \frac{9}{2}\right)$$

$$\text{Fila 3: } 3(7, -1, 2, 9) - \frac{1}{2}(0, -1, -3, 0) + 1(0, 0, 5, 2) = \left(21, -\frac{5}{2}, \frac{25}{2}, 29\right)$$

$$\text{Fila 4: } -4(7, -1, 2, 9) + 1(0, -1, -3, 0) + 1(0, 0, 10, 4) = (-28, 3, -1, -32)$$

Así, la matriz $\mathbf{A}$ expresada explícitamente es:

$$\mathbf{A} = \begin{pmatrix} 7 & -1 & 2 & 9 \\ \frac{7}{2} & -\frac{3}{2} & -2 & \frac{9}{2} \\ 21 & -\frac{5}{2} & \frac{25}{2} & 29 \\ -28 & 3 & -1 & -32 \end{pmatrix}$$

(b) Justificar si la factorización LU es única

Solución: Dado que las operaciones elementales para obtener $\mathbf{U}$ a partir de $\mathbf{A}$ no requirieron intercambio de filas, podemos calcular el determinante de $\mathbf{A}$ directamente a partir de la factorización $\mathbf{A} = \mathbf{L}\mathbf{U}$:

$$\det(\mathbf{A}) = \det(\mathbf{L}) \cdot \det(\mathbf{U})$$

Como $\mathbf{L}$ es triangular unitaria, $\det(\mathbf{L}) = 1$. Para la matriz triangular superior $\mathbf{U}$, el determinante es el producto de los elementos de su diagonal principal:

$$\det(\mathbf{U}) = 7 \cdot (-1) \cdot 5 \cdot 4 = -140$$

Por lo tanto, $\det(\mathbf{A}) = -140 \neq 0$, concluyendo que $\mathbf{A}$ es no singular (todas sus menores principales son no nulas). Se garantiza entonces que $\mathbf{A}$ posee una única factorización LU con L unitaria.

(c) Resolver el sistema lineal $\mathbf{Ax} = \mathbf{b}$

Dado $\mathbf{b} = (31, \frac{3}{2}, 124, -90)^t$, resolvemos el sistema en dos partes empleando la descomposición $\mathbf{LUx} = \mathbf{b}$:

Paso 1: Resolver $\mathbf{Ly} = \mathbf{b}$ mediante sustitución hacia adelante

El sistema matricial $\mathbf{Ly} = \mathbf{b}$ se expresa como:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 \\ 3 & -\frac{1}{2} & 1 & 0 \\ -4 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 31 \\ \frac{3}{2} \\ 124 \\ -90 \end{pmatrix}$$

Despejando de arriba hacia abajo:

Ecuación 1 : $y_1 = 31$

Ecuación 2 : $\frac{1}{2}y_1 + y_2 = \frac{3}{2} \implies y_2 = \frac{3}{2} - \frac{31}{2} \implies y_2 = -14$

Ecuación 3 : $3y_1 - \frac{1}{2}y_2 + y_3 = 124$
 $\implies 3(31) - \frac{1}{2}(-14) + y_3 = 124$
 $\implies 93 + 7 + y_3 = 124 \implies y_3 = 124 - 100 \implies y_3 = 24$

Ecuación 4 : $-4y_1 + y_2 + y_4 = -90$
 $\implies -4(31) + (-14) + y_4 = -90$
 $\implies -124 - 14 + y_4 = -90 \implies y_4 = -90 + 138 \implies y_4 = 48$

Obtenemos así el vector:

$$\mathbf{y} = \begin{pmatrix} 31 \\ -14 \\ 24 \\ 48 \end{pmatrix}$$

Paso 2: Resolver $\mathbf{Ux} = \mathbf{y}$ mediante sustitución hacia atrás

El sistema matricial $\mathbf{Ux} = \mathbf{y}$ se expresa como:

$$\begin{pmatrix} 7 & -1 & 2 & 9 \\ 0 & -1 & -3 & 0 \\ 0 & 0 & 5 & 2 \\ 0 & 0 & 10 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 31 \\ -14 \\ 24 \\ 48 \end{pmatrix}$$

Analizando las ecuaciones de las filas 3 y 4:

$$(3) \quad 5x_3 + 2x_4 = 24$$

$$(4) \quad 10x_3 + 4x_4 = 48$$

Note que la ecuación (4) es linealmente dependiente de la (3), por lo que el sistema posee infinitas soluciones. Defina $x_4 = t \in \mathbb{R}$ despejando de abajo hacia arriba:

$$\text{Despeje de } x_3 : 5x_3 + 2t = 24 \implies 5x_3 = 24 - 2t \implies x_3 = \frac{24 - 2t}{5}$$

$$\begin{aligned} \text{Despeje de } x_2 : -x_2 - 3x_3 &= -14 \implies x_2 = 14 - 3x_3 \\ \implies x_2 &= 14 - 3\left(\frac{24 - 2t}{5}\right) \\ \implies x_2 &= \frac{70 - 72 + 6t}{5} \implies x_2 = \frac{-2 + 6t}{5} \end{aligned}$$

$$\begin{aligned} \text{Despeje de } x_1 : 7x_1 - x_2 + 2x_3 + 9x_4 &= 31 \\ \implies 7x_1 &= 31 + x_2 - 2x_3 - 9t \\ \implies 7x_1 &= 31 + \left(\frac{-2 + 6t}{5}\right) - 2\left(\frac{24 - 2t}{5}\right) - 9t \\ \implies 7x_1 &= \frac{155 + (-2 + 6t) - (48 - 4t) - 45t}{5} \\ \implies 7x_1 &= \frac{105 - 35t}{5} = 21 - 7t \\ \implies x_1 &= \frac{21 - 7t}{7} \implies x_1 = 3 - t \end{aligned}$$

Conjunto Solución Final

El conjunto solución del sistema $\mathbf{Ax} = \mathbf{b}$ escrito en forma vectorial paramétrica es:

$$S = \left\{ \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \in \mathbb{R}^4 \mid \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 3 \\ -\frac{2}{5} \\ \frac{24}{5} \\ 0 \end{pmatrix} + t \begin{pmatrix} -1 \\ \frac{6}{5} \\ -\frac{2}{5} \\ 1 \end{pmatrix}, t \in \mathbb{R} \right\}$$

6. Ejercicio 6

Enunciado: Considere la siguiente matriz

$$A = \begin{pmatrix} -\frac{1}{2} & \frac{3}{2} & 5 \\ \frac{1}{2} & -\frac{3}{2} & -2 \\ \frac{1}{2} & \frac{7}{2} & 1 \\ \frac{1}{2} & \frac{7}{2} & 4 \end{pmatrix}$$

(a) Encuentre la matriz Q ortogonal y R triangular superior tales que $A = QR$

Solución:

Se desea llegar a la descomposición $A = QR$ donde Q se obtiene al multiplicar múltiples matrices Q_i , se procede a calcular las matrices Q_i

Paso 1: Tome $z = \left[\frac{-1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{-1}{2} \right]^t$, con este vector es posible calcular v_1 con el cuál se va a construir la matriz Q_1 .

El vector v_1 se calcula con la siguiente formula:

$$v_1 = \text{sign}(z_1) \|z\|_2 e_1 + z$$

Donde e_1 es el primer vector canonico, es decir, tiene un 1 en su primera entrada y ceros en las demás, esté cambia de tamaño dependiendo del tamaño de z . Con esta formula se tiene que:

$$v_1 = \left[\frac{-3}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right]^t$$

Con el v_1 se puede calcular la primera matriz F_1 que sera necesaria para encontrar Q_1 con la siguiente expresión:

$$F_1 = I - 2 \frac{v_1 v_1^t}{v_1^t v_1}$$

Donde I corresponde a la matriz identidad del tamaño necesario. Entonces:

$$v_1 v_1^t = \begin{pmatrix} \frac{9}{4} & -\frac{3}{4} & -\frac{3}{4} & -\frac{3}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{pmatrix}$$

$$v_1^t v_1 = 3$$

$$F_1 = I - \frac{2}{3} \begin{pmatrix} \frac{9}{4} & -\frac{3}{4} & -\frac{3}{4} & -\frac{3}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{pmatrix} = \begin{pmatrix} -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{5}{6} & -\frac{1}{6} & -\frac{1}{6} \\ \frac{1}{2} & -\frac{1}{6} & \frac{5}{6} & -\frac{1}{6} \\ \frac{1}{2} & -\frac{1}{6} & -\frac{1}{6} & \frac{5}{6} \end{pmatrix}$$

La matriz Q_i se construye de la siguiente manera:

$$Q_j = \begin{bmatrix} I_j & 0 \\ 0 & F_j \end{bmatrix}_{m \times m}.$$

En este primer paso, se tiene que $Q_1 = F_1$.

Para finalizar, se calcula la matriz que se utilizara en el siguiente paso:

$$Q_1 A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & -\frac{5}{3} & 0 \\ 0 & \frac{10}{3} & 3 \\ 0 & \frac{10}{3} & 6 \end{pmatrix}$$

Paso 2: Tome $z = \left[\frac{-5}{3}, \frac{10}{3}, \frac{10}{3} \right]^t$ y construya el v_2

$$v_2 = \text{sign}(z_1) \|z\|_2 e_2 + z$$

$$v_2 = \left[\frac{-20}{3}, \frac{10}{3}, \frac{10}{3} \right]^t$$

Con el v_2 se puede calcular la segunda matriz F_2 con la siguiente expresión:

$$F_2 = I - 2 \frac{v_2 v_2^t}{v_2^t v_2}$$

Entonces:

$$v_2 v_2^t = \begin{pmatrix} \frac{400}{9} & -\frac{200}{9} & -\frac{200}{9} \\ -\frac{200}{9} & \frac{100}{9} & \frac{100}{9} \\ -\frac{200}{9} & \frac{100}{9} & \frac{100}{9} \end{pmatrix}$$

$$v_2^t v_2 = \frac{200}{3}$$

$$F_2 = I - \frac{3}{100} \begin{pmatrix} \frac{400}{9} & -\frac{200}{9} & -\frac{200}{9} \\ -\frac{200}{9} & \frac{100}{9} & \frac{100}{9} \\ -\frac{200}{9} & \frac{100}{9} & \frac{100}{9} \end{pmatrix} = \begin{pmatrix} -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

Con F_2 se puede construir la matriz Q_2

$$Q_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \\ 0 & \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ 0 & \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

Para finalizar, se calcula la matriz que se utilizara en el siguiente paso:

$$Q_1 Q_2 A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 5 & 6 \\ 0 & 0 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

Paso 3: Tome $z = [0, 3]^t$ y construya el v_3

$$v_3 = \text{sign}(z_1) \|z\|_2 e_3 + z$$

$$v_3 = [3, 3]^t$$

Con el v_3 se puede calcular la tercer matriz F_3 con la siguiente expresión:

$$F_3 = I - 2 \frac{v_3 v_3^t}{v_3^t v_3}$$

Entonces:

$$v_3 v_3^t = \begin{pmatrix} 9 & 9 \\ 9 & 9 \end{pmatrix}$$

$$v_3^t v_3 = 18$$

$$F_2 = I - \frac{1}{9} \begin{pmatrix} 9 & 9 \\ 9 & 9 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix}$$

Con F_3 se puede construir la matriz Q_3

$$Q_3 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Con las tres matrices se puede calcular la matriz Q

$$Q = Q_1 Q_2 Q_3 = \begin{pmatrix} -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

Como $QR = A$ y A es conocido, entonces podría encontrar la matriz R .

$$R = Q^{-1}A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 5 & 6 \\ 0 & 0 & -3 \\ 0 & 0 & 0 \end{pmatrix}$$

Entonces, la descomposición de A en QR resulta en las dos matrices:

$$Q = \begin{pmatrix} -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

$$R = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 5 & 6 \\ 0 & 0 & -3 \\ 0 & 0 & 0 \end{pmatrix}$$

(b) Resuelva el sistema $Ax = b$ donde $b = (7, -1, 0, 6)^t$

Solución:

Para resolver el sistema $Ax = b$, recuerde que $A = QR$ y entonces:

$$QRx = b$$

$$Rx = Q^{-1}b$$

$$\begin{pmatrix} 1 & 2 & -1 \\ 0 & 5 & 6 \\ 0 & 0 & -3 \\ 0 & 0 & 0 \end{pmatrix} x = \begin{pmatrix} -1 \\ 7 \\ -6 \\ 0 \end{pmatrix}$$

Se obtiene el siguiente sistema de ecuaciones:

$$\begin{cases} x_1 + 2x_2 - x_3 = -1 \\ 5x_2 + 6x_3 = 7 \\ -3x_3 = -6 \\ 0 = 0 \end{cases}$$

De la tercera ecuación, se obtiene que $x_3 = 2$. Con esto, se puede obtener el valor de x_2 en la segunda ecuación:

$$\begin{aligned} 5x_2 &= 7 - 6(2) = -5 \\ x_2 &= -1 \end{aligned}$$

Y de la primera ecuación se obtiene el valor de x_1

$$\begin{aligned} x_1 + 2(-1) - 2 &= -1 \\ x_1 &= 3 \end{aligned}$$

Entonces, la solución del sistema $Ax = b$ será:

$$x = [3, -1, 2]^t$$

7. Ejercicio 7

Sea $p(x)$ un polinomio de grado n , se dice que x_0 es una raíz múltiple de $p_n(x)$ si $p_n(x) = (x - x_0)^k p_{n-k}(x)$, con $k > 1$ y $p_{n-k}(x)$ un polinomio de grado $n - k$. Por ejemplo, $p_4(x) = x^4 - 6x^3 + 12x^2 - 10x + 3$ tiene una raíz múltiple $x_0 = 1$, ya que $p_4(x) = (x - 1)^3(x - 3)$. Para este tipo de polinomios el método de Newton tiene una convergencia lenta, por este motivo se introduce la siguiente modificación en su iteración, para p un polinomio:

$$x_{k+1} = x_k - \frac{p(x_k)p'(x_k)}{[p'(x_k)]^2 - p(x_k)p''(x_k)}$$

(a) ¿Qué condiciones se necesitan para que el método funcione? Explique su respuesta.

Para que el método modificado de Newton para raíces múltiples funcione correctamente y esté bien definido, se requieren de las siguientes condiciones:

- El polinomio $p(x)$ debe ser al menos de clase C^2 en el intervalo en cuestión.

- En cada iteración k , el denominador tiene que ser distinto a 0: .

$$[p'(x_k)]^2 - p(x_k)p''(x_k) \neq 0$$

- El valor inicial debe elegirse suficientemente cerca de la raíz múltiple para garantizar la convergencia a dicha raíz. En caso contrario el algoritmo puede converger a una raíz distinta.

(b) Escriba un Algoritmo del método descrito anteriormente.

1. **Entradas:** El polinomio $p(x)$, la primera derivada $p'(x)$, la segunda derivada $p''(x)$, el punto inicial x_0 , la tolerancia ε y el número máximo de iteraciones `iterMax`.
2. **Inicialización:** Hacer $k = 1$, $x_{\text{ant}} = x_0$ y error = 0.
3. **Bucle iterativo:** Mientras error $> \varepsilon$ y $k \leq \text{iterMax}$ hacer:
 - Calcular el denominador: $[p'(x_{\text{ant}})]^2 - p(x_{\text{ant}})p''(x_{\text{ant}})$.
 - Si es 0, entonces interrumpir la ejecución con un mensaje de error.
 - Calcular la nueva aproximación:

$$x_{\text{act}} = x_{\text{ant}} - \frac{p(x_{\text{ant}})p'(x_{\text{ant}})}{[p'(x_{\text{ant}})]^2 - p(x_{\text{ant}})p''(x_{\text{ant}})}$$

- Calcular el error absoluto: error = $|x_{\text{act}} - x_{\text{ant}}|$.
 - Actualizar los valores: $x_{\text{ant}} = x_{\text{act}}$ y $k = k + 1$.
4. **Salida:** Devolver la aproximación final x_{act} y el historial de iteraciones.

(c) Considere el polinomio y realice las iteraciones solicitadas.

Enunciado: Considere el polinomio:

$$p(x) = x^6 - 12x^5 + 63x^4 - 216x^3 + 567x^2 - 972x + 729 = (x^2 + 9)(x - 3)^4$$

con el punto inicial $x_0 = 0$. Realice 4 iteraciones del método y encuentre en cada una el error absoluto. Describa el resultado encontrado.

Sea $x_0 = 0$ para encontrar la raíz múltiple $r = 3$ (de multiplicidad $m = 4$), a continuación se muestran los resultados de las primeras 4 iteraciones y sus respectivos errores absolutos $e_k = |x_k - 3|$:

Cuadro 2: Iteraciones del método modificado para $p(x) = (x^2 + 9)(x - 3)^4$

Iteración (k)	Aproximación (x_k)	Error Absoluto ($ x_k - 3 $)
0	0	3,0
1	2,2500000000	0,7500000000
2	2,9662921348	0,0337078652
3	2,9999581177	0,0000418823
4	2,9999999999	$6,51 \times 10^{-11}$

(d) Escriba una función de MATLAB la cual lleve a cabo la implementación del método.

Listing 2: Función en MATLAB para el Método de Newton Modificado en Raíces Múltiples

```
1 function M = metodoNewtonModificadoMultiples(p_coef, df_coef, ddf_coef,
2     x0, tol, iterMax)
3     % METODONEWTOMODIFICADORAIZMULTIPLE Implementa el algoritmo de
4     % Newton para encontrar raices multiples de un polinomio
5     %
6     % Entradas:
7     % p_coef - Vector de coeficientes del polinomio p(x)
8     % df_coef - Vector de coeficientes de la primera derivada p'(x)
9     % ddf_coef - Vector de coeficientes de la segunda derivada p''(x)
10    % x0 - Aproximacion inicial
11    % tol - Tolerancia
12    % iterMax - Numero maximo de iteraciones
13    %
14    % Salida:
15    % M - Matriz de resultados
16
17    k = 1;
18    xk_ant = x0;
19
20    px = polyval(p_coef, xk_ant);
21    dpx = polyval(df_coef, xk_ant);
22    ddpdx = polyval(ddf_coef, xk_ant);
23
24    den = (dpx)^2 - px * ddpdx;
25
26    if abs(den) < eps
27        error('Error: El denominador es 0.');
```

```
28    end
29
30    M = zeros(iterMax + 1, 3);
31    M(1, :) = [0, xk_ant, inf];
32
33    error_abs = tol + 1;
34
35    while error_abs > tol && k <= iterMax
36        xk_act = xk_ant - (px * dpx) / den;
37        error_abs = abs(xk_act - xk_ant);
38
39        M(k + 1, :) = [k, xk_act, error_abs];
40
41        xk_ant = xk_act;
42        px = polyval(p_coef, xk_ant);
43        dpx = polyval(df_coef, xk_ant);
```

```

42     ddpdx = polyval(ddf_coef, xk_ant);
43
44     den = (dpx)^2 - px * ddpdx;
45
46     if abs(den) < eps
47         fprintf('Error: El denominador se anulo en la iteracion %d
48             .\n', k);
49         M = M(1:k+1, :);
50         return;
51     end
52
53     k = k + 1;
54
55     M = M(1:k, :);
56 end

```

8. Ejercicio 8

Enunciado: (Intervalos de convergencia del método de Newton) Considere la función

$$f(x) = 816x^3 - 3835x^2 + 6000x - 3125$$

La cual tiene tres raíces reales diferentes contenidas en el intervalo $[1,4, 1,7]$.

(a) Utilice la función `fzero` de MATLAB para aproximar las tres soluciones. (Para esta función es posible encontrar las tres raíces de manera exacta pues son números racionales).

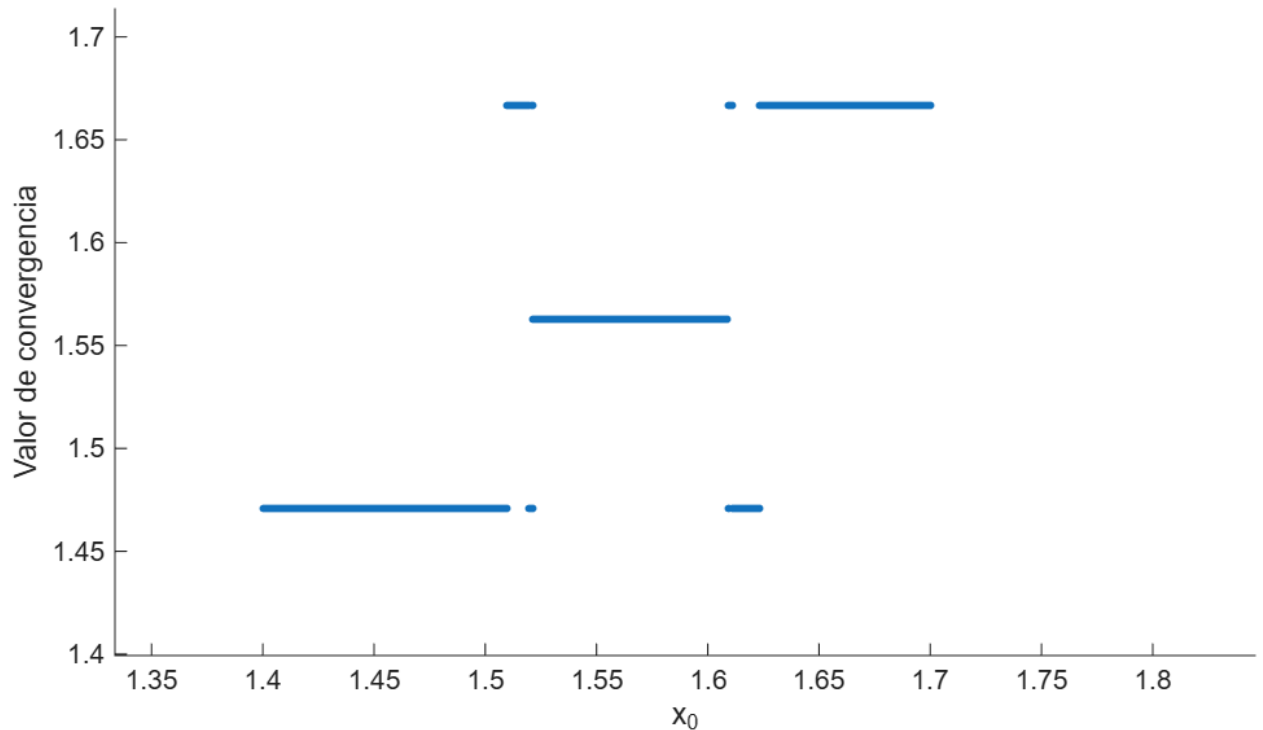
Utilizando la función `fzero` las raíces son (formato long):

$$x_1 = 1,6666666666666624$$

$$x_2 = 1,470588235294116$$

$$x_3 = 1,5625000000000031$$

(b) Defina un vector de valores iniciales $x_0 = \text{linspace}(1,4,1,7)$. Para cada elemento de x_0 obtenga el valor de convergencia del método de Newton. Grafique el valor de convergencia en función del valor inicial.



(c) La base de atracción se refiere al mayor intervalo I_c que contiene a un cero c , que garantiza que el método convergerá a c , si tomamos cualquier elemento del intervalo como valor inicial. Teóricamente, para la convergencia es necesario que $|x_0 - c| \leq \left[\frac{f''(c)}{2f'(c)} \right]^{-1}$. Compare este estimado con los resultados numéricos de la parte (b). ¿Hay algún estimado particularmente malo? En caso afirmativo, ¿puede explicar el porqué?

I_{x_1} :

$$|x_0 - x_1| = \left| \frac{f''(x_1)}{2f'(x_1)} \right|^{-1}$$

$$|x_0 - x_1| = \frac{2045}{147}$$

$$\frac{-600}{49} \leq x_0 \leq \frac{2290}{147}$$

$$-12,2448 \leq x_0 \leq 15,5782$$

Para x_2

$$-9,8248 \leq x_0 \leq 12,7660$$

Para x_3

$$-296,8750 \leq x_0 \leq 300$$

Los intervalos parecen ser extremadamente grandes comparados con las distancias entre las raíces, además unos están contenidos dentro de otros, entre 1,5 y 1,55 el método se vuelve muy sensible y oscila mucho entre las raíces, igualmente para los x_0 entre 1,6 y 1,65, aproximadamente.

9. Ejercicio 9

Enunciado: (Método de Halley) Dada una función $f \in C^2([a, b])$ y un valor inicial x_0 , define la sucesión $\{x_k\}_{k=0}^{\infty}$ dada por:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \left[1 - \frac{f(x_k)}{f'(x_k)} \cdot \frac{f''(x_k)}{2f'(x_k)} \right]^{-1}$$

(a) Existencia de $\delta > 0$ tal que $x_k \rightarrow c$

Solución:

Para analizar la iteración punto fijo $x_{k+1} = g(x_k)$, definimos la función:

$$g(x) = x - \frac{f(x)}{f'(x)} \left[1 - \frac{f(x)f''(x)}{2(f'(x))^2} \right]^{-1}$$

Simplificando tenemos $g(x)$ como:

$$g(x) = x - \frac{2f(x)f'(x)}{2(f'(x))^2 - f(x)f''(x)}$$

Hipótesis requeridas:

- I. $f \in C^3([a, b])$, lo cual garantiza que $g(x)$ sea de clase C^2 en un entorno alrededor de c .
- II. $f'(c) \neq 0$ (c es un cero simple), asegurando que el denominador no sea 0 en $x = c$.

Demostración de convergencia local: Evaluamos la función en el punto c donde $f(c) = 0$:

$$g(c) = c - \frac{2f(c)f'(c)}{2(f'(c))^2 - f(c)f''(c)} = c - 0 = c$$

Por lo que c es un punto fijo de g . Ahora se calcula la primera derivada de $g(x)$. Escribiendo $g(x) = x - \frac{u(x)}{v(x)}$ con $u(x) = 2f(x)f'(x)$ y $v(x) = 2(f'(x))^2 - f(x)f''(x)$:

$$g'(x) = 1 - \frac{u'(x)v(x) - u(x)v'(x)}{[v(x)]^2}$$

Evaluando en $x = c$ (con $f(c) = 0$):

$$u(c) = 0$$

$$u'(c) = 2(f'(c))^2 + 2f(c)f''(c) = 2(f'(c))^2$$

$$v(c) = 2(f'(c))^2 - 0 = 2(f'(c))^2$$

Sustituyendo estos valores en $g'(c)$:

$$g'(c) = 1 - \frac{2(f'(c))^2 \cdot 2(f'(c))^2 - 0}{[2(f'(c))^2]^2} = 1 - 1 = 0$$

Como $|g'(c)| = 0 \leq L < 1$ entonces g es una contracción sobre el intervalo $[a, b]$, por el teorema del punto fijo existe un entorno $(c - \delta, c + \delta) \subseteq [a, b]$ con $\delta > 0$ tal que para cualquier valor inicial $x_0 \in [a, b]$, la sucesión $\{x_k\}_{k=0}^{\infty}$ converge a c .

(b) Demostración de que la velocidad de convergencia es cúbica

Solución:

Por definición, una sucesión $\{x_k\}_{k=0}^{\infty}$ que converge a c posee velocidad de convergencia cúbica (orden $q = 3$) si existe $\mu \geq 0$ tal que:

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^3} = \mu < \infty, \quad \text{donde } e_k = x_k - c$$

En términos del error e_k :

$$e_{k+1} = e_k - \frac{2f(x_k)f'(x_k)}{2(f'(x_k))^2 - f(x_k)f''(x_k)} = \frac{e_k [2(f'(x_k))^2 - f(x_k)f''(x_k)] - 2f(x_k)f'(x_k)}{2(f'(x_k))^2 - f(x_k)f''(x_k)}$$

Dado que $f \in C^3([a, b])$, podemos desarrollar $f(x_k)$, $f'(x_k)$ y $f''(x_k)$ usando expansiones de Taylor de grado 3 alrededor de c (donde $f(c) = 0$):

$$f(x_k) = f'(c) [e_k + c_2 e_k^2 + c_3 e_k^3 + O(e_k^4)]$$

$$f'(x_k) = f'(c) [1 + 2c_2 e_k + 3c_3 e_k^2 + O(e_k^3)]$$

$$f''(x_k) = f'(c) [2c_2 + 6c_3 e_k + O(e_k^2)]$$

con los términos $c_2 = \frac{f''(c)}{2f'(c)}$ y $c_3 = \frac{f'''(c)}{6f'(c)}$.

Sustituyendo los desarrollos de Taylor en e_{k+1} tenemos:

$$\begin{aligned} e_{k+1} &= \frac{e_k [2 + 6c_2 e_k + (6c_2^2 + 6c_3) e_k^2 + O(e_k^3)] - 2(e_k + c_2 e_k^2 + c_3 e_k^3) (1 + 2c_2 e_k + 3c_3 e_k^2) + O(e_k^4)}{2 + 6c_2 e_k + O(e_k^2)} \\ &= \frac{[2e_k + 6c_2 e_k^2 + (6c_2^2 + 6c_3) e_k^3] - [2e_k + 6c_2 e_k^2 + (4c_2^2 + 8c_3) e_k^3] + O(e_k^4)}{2 + 6c_2 e_k + O(e_k^2)} \\ &= \frac{2(c_2^2 - c_3) e_k^3 + O(e_k^4)}{2 + 6c_2 e_k + O(e_k^2)} \end{aligned}$$

Tomando el límite cuando $k \rightarrow \infty$ ($e_k \rightarrow 0$) y sacando factor común de e_k^3 para evitar la indeterminación $0/0$:

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^3} = \lim_{e_k \rightarrow 0} \left| \frac{e_k^3 [2(c_2^2 - c_3) + O(e_k)]}{e_k^3 [2 + O(e_k)]} \right| = \lim_{e_k \rightarrow 0} \left| \frac{2(c_2^2 - c_3) + O(e_k)}{2 + O(e_k)} \right| = |c_2^2 - c_3| = \mu$$

Donde la constante μ está dada por:

$$\mu = \left| \left(\frac{f''(c)}{2f'(c)} \right)^2 - \frac{f'''(c)}{6f'(c)} \right| < \infty$$

Puesto que $f'(c) \neq 0$, la constante μ existe y es finita, por lo que la velocidad de convergencia es cúbica ($q = 3$).

(c) Implementación de la función `iter`

Enunciado: Escriba una función `suc = iter(f, x0, tol)` en MATLAB que calcule la sucesión definida anteriormente, donde las entradas son la función f , el valor inicial x_0 y cierta tolerancia tol . Por simplicidad en la implementación, asuma que f es un polinomio, por lo que al calcular sus derivadas podemos hacerlo de manera exacta (a precisión de la máquina).

Código de la Función `iter(iter.m)`

Listing 3: Función para el Método de Halley

```
1 function suc = iter(f, x0, tol)
2     % iter: Calcula la sucesion de aproximaciones mediante el Metodo de
3     % Halley
4     % Entradas:
5     %   f - Vector de coeficientes del polinomio [a_n, a_{n-1}, ...,
6     %       a_1, a_0]
7     %   x0 - Valor inicial de la sucesion (x_0)
8     %   tol - Tolerancia
9     % Salida:
10    %   suc - Vector columna con la sucesion de valores [x0, x1, x2,
11    %       ...]
12    df = polyder(f);
13    ddf = polyder(df);
14    suc = x0;
15    xk = x0;
16    error = inf;
17    while error > tol
18        fx = polyval(f, xk);
19        dfx = polyval(df, xk);
20        ddfx = polyval(ddf, xk);
21        xk_nuevo = xk - (fx/dfx) * (1 / (1 - (fx/dfx) * (ddfx / (2*dfx))));
22        error = abs(xk_nuevo - xk);
23        suc = [suc; xk_nuevo];
24        xk = xk_nuevo;
25    end
26 end
```

(d) Verificación y Comparación con el Método de Newton

Enunciado: Verifique el comportamiento de su función para el polinomio $f(x) = 816x^3 - 3835x^2 + 6000x - 3125$, utilizando $x_0 = 1,6$, con una tolerancia $tol = 10^{-6}$. Compare el comportamiento de la sucesión obtenida con la que se obtiene al aplicar el Método de Newton. Dibuje en un mismo gráfico el error $|x_k - c|$ para ambos casos. Comente sus resultados.

Implementación del Método de Newton (metodoNewton.m)

Para el método de Newton se usó la función realizada en clase con leves modificaciones:

Listing 4: Función metodoNewton.m

```

1 function[M] = metodoNewton(f,df,x,tol)
2     cont = 1;
3     er = tol + 1; M(1,1) = x;
4     M(1,2) = inf;
5     while er > tol
6         t = x;
7         q = df(x);
8         if (abs(q)<tol)
9             return;
10        end
11        x = x - f(x)/q;
12        er = abs(t-x);
13
14        M(cont+1,1) = x;
15        M(cont+1,2) = er;
16        cont = cont + 1;
17    end
18 end

```

Tabla Comparativa de Iteraciones y Errores Exactos

Para la función polinomial $f(x) = 816x^3 - 3835x^2 + 6000x - 3125$, la raíz real exacta es $c = \frac{3125}{2000} = 1,5625$. Evaluando ambos métodos con $x_0 = 1,6$ y tolerancia $\text{tol} = 10^{-6}$, se obtienen los siguientes valores obtenidos de la ejecución en MATLAB:

Cuadro 3: Comparación de errores entre el Método de Halley y el Método de Newton

k	Método de Halley		Método de Newton	
	Aproximación (x_k)	Error $ x_k - c $	Aproximación (x_k)	Error $ x_k - c $
0	1,6000000000000000	$3,750000 \times 10^{-2}$	1,6000000000000000	$3,750000 \times 10^{-2}$
1	1,571727853820200	$9,227854 \times 10^{-3}$	1,548437500000150	$1,406250 \times 10^{-2}$
2	1,562581921078840	$8,192108 \times 10^{-5}$	1,563424628063110	$9,246281 \times 10^{-4}$
3	1,562500000058180	$5,818368 \times 10^{-11}$	1,562500927240930	$9,272409 \times 10^{-7}$
4	1,562500000000030	$3,419487 \times 10^{-14}$	1,562500000001150	$1,148193 \times 10^{-12}$

Gráfico Comparativo del Error

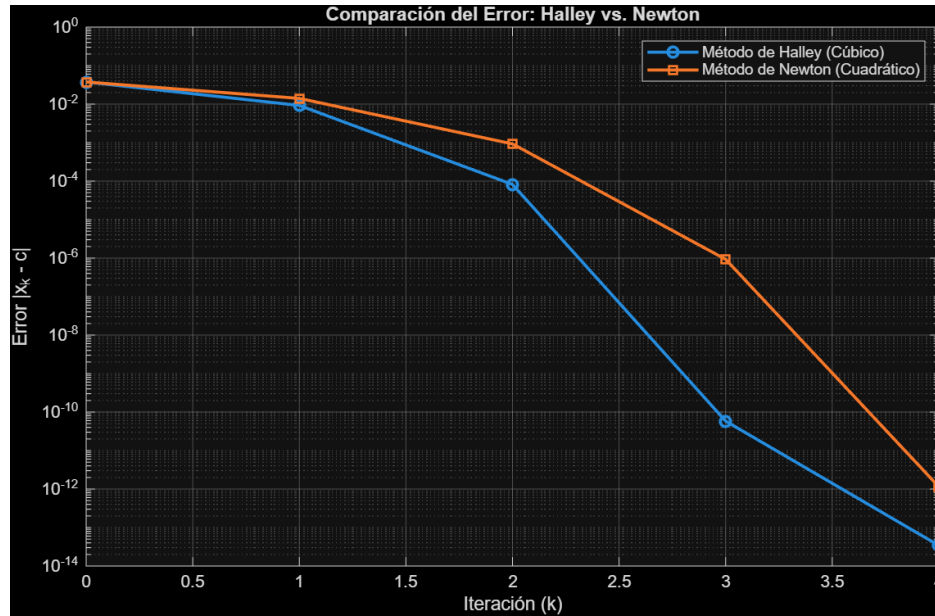


Figura 3: Gráfico de decaimiento del error $|x_k - c|$ en escala semilogarítmica para los Métodos de Halley y Newton.

Ambos métodos convergen a la raíz verdadera $c = 1,5625$ partiendo de $x_0 = 1,6$. Pero como se puede observar en la figura 3 la curva semilogarítmica correspondiente al error decrece en mayor medida en cada iteración en el caso de Halley a comparación de Newton. Este comportamiento es consistente con el orden de convergencia de ambos métodos. Mientras que el Método de Newton tiene convergencia cuadrática ($p = 2$), el Método de Halley posee una convergencia cúbica ($p = 3$).

(e) Intervalos de Atracción del Método de Halley

Enunciado: Grafique los intervalos de atracción del Método de Halley. Para ello, considere el vector de valores iniciales $xx = \text{linspace}(1.4, 1.7)$. Grafique el valor de convergencia del método en función de xx . ¿Existe alguna mejoría con respecto al Método de Newton?

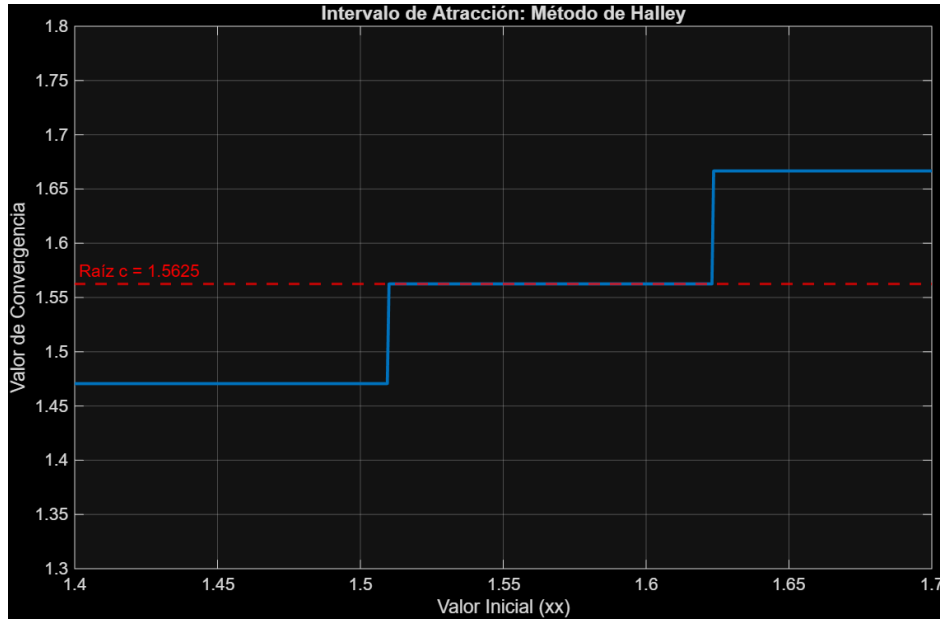


Figura 4: Intervalo de atracción para el Método de Halley evaluado en el rango $x \in [1,4,1,7]$.

El polinomio $f(x) = 816x^3 - 3835x^2 + 6000x - 3125$ posee tres raíces reales distintas en el intervalo analizado:

$$c_1 = \frac{25}{17} \approx 1,470588, \quad c_2 = \frac{125}{16} = 1,5625, \quad c_3 = \frac{5}{3} \approx 1,666667$$

A partir de la Figura 4, se identifican con precisión a que raíz converge el metodo dependiendo del valor inicial x_0 :

- Para $x_0 \in [1,40, 1,51)$, el método converge hacia $c_1 \approx 1,4706$.
- Para $x_0 \in [1,51, 1,62]$, el método converge exactamente a la raíz central $c_2 = 1,5625$.
- Para $x_0 \in (1,62, 1,70]$, el método converge hacia $c_3 \approx 1,6667$.

Se observa una mejoría sustancial del Método de Halley respecto al Método de Newton. Halley define tres regiones de atracción continuas en forma de escalón, mientras que Newton suele volverse caótico en las fronteras o zonas donde $f'(x) \approx 0$. Al incorporar la segunda derivada $f''(x)$, Halley compensa la curvatura de la función, evitando que los pasos intermedios sean proyectados a regiones lejanas lo que garantiza una convergencia más predecible y estable.

10. Ejercicio 10

Enunciado: La razón de cambio de temperatura en el tiempo t de un objeto es proporcional a la diferencia entre la temperatura ambiente T_a y la temperatura T del objeto, es decir

$$\frac{dT}{dt} = k(T_a - T)$$

Donde k se le llama constante de proporcionalidad.

(a) Verifique que la temperatura del objeto en el tiempo t viene dada por $T(t) = T_a + (T_0 - T_a)e^{-kt}$.

Solución:

Partiendo de lo brindado en el enunciado se puede ver que:

$$\frac{dT(t)}{dt} = \frac{d(T_a + (T_0 - T_a)e^{-kt})}{dt}$$

Y, como T_0 y T_a son valores constantes, entonces se tiene que:

$$\frac{dT(t)}{dt} = -k(T_0 - T_a)e^{-kt}$$

Note que $(T_0 - T_a)e^{-kt} = T(t) - T_a$, por lo tanto:

$$\frac{dT(t)}{dt} = -k(T_0 - T_a)e^{-kt} = -k(T(t) - T_a) = k(T_a - T(t))$$

$$\rightarrow \frac{dT(t)}{dt} = k(T_a - T(t))$$

Y así, se llega a la definición de la Ley de Enfriamiento de Newton del enunciado.

(b) Un médico forense desea estimar el valor de k para poder determinar el tiempo de muerte de cadáveres. Para estimar k , el médico considera minimizar el funcional $f(k) := \sum_i (T(t_i) - T_i)^2$. Aproxime el punto crítico de f y verifique que f alcanza un mínimo.

Solución:

En el enunciado se brinda la siguiente tabla de valores:

t_i (horas)	T_i (°C)
0.0	37.00
0.2	36.72
0.4	36.41
0.6	36.12
0.8	35.90

De la tabla se infiere que el valor $T_0 = 37^\circ$ y se menciona que la temperatura ambiente es de 21° . Recuerde la definición de $T(t)$ del inciso anterior y vea que:

$$f(k) = \sum_i \left(T_a + (T_0 - T_a)e^{-kt_i} - T_i \right)^2$$

Para encontrar el punto crítico de la función se debe primero calcular su primera derivada, pues el punto crítico k_c debe cumplir que $f'(k_c) = 0$. Considere $g_i(k) = T_a + (T_0 - T_a)e^{-kt_i} - T_i$, entonces:

$$f(k) = \sum_i g_i(k)^2$$

$$f'(k) = \sum_i 2 \cdot g_i(k) \cdot g'_i(k)$$

$$f'(k) = -2(T_0 - T_a) \sum_i g_i(k)(t_i e^{-kt_i})$$

Como T_0 y T_a son dados entonces:

$$f'(k) = -2(37 - 21) \sum_i g_i(k)(t_i e^{-kt_i}) = -32 \sum_i g_i(k)(t_i e^{-kt_i})$$

$$\rightarrow f'(k) = -32 \sum_i (21 + 16e^{-kt_i} - T_i)t_i e^{-kt_i}$$

La función $f(k)$ y sus derivadas se definen en MATLAB y con ayuda de la función `fzero()` se puede aproximar el punto crítico k_c tal que $f'(k_c) = 0$ con esto se obtiene que $k_c \approx 0,0913$ es el punto crítico de la función.

Para determinar si este punto es un mínimo, se procede a calcular la segunda derivada y evaluar en k_c

$$f''(k) = \left[-32 \sum_i g_i(k)(t_i e^{-kt_i}) \right]' = -32 \sum_i t_i [g_i(k)]' e^{-kt_i} + t_i [e^{-kt_i}]' g_i(k)$$

$$f''(k) = -32 \sum_i (t_i \cdot (-16 \cdot t_i e^{-kt_i}) e^{-kt_i} - t_i \cdot t_i e^{-kt_i} (21 + 16e^{-kt_i} - T_i))$$

$$\rightarrow f''(k) = \sum_i (512 \cdot t_i^2 e^{-2kt_i} + 32t_i^2 e^{-kt_i} (21 + 16e^{-kt_i} - T_i))$$

Al evaluar lo anterior en k_c se obtiene que:

$$f''(k_c) \approx 544,1009$$

Como el valor obtenido es mayor que cero entonces se puede concluir que la función $f(k)$ alcanza un mínimo en su punto crítico.

(c) Un día caluroso en San Pedro a 31°C una persona falleció. Al llegar al lugar de los hechos, se determina que la temperatura del cuerpo es de 34°C . ¿Hace cuánto falleció la persona?

Solución:

Recuerde la definición de la temperatura de un objeto dado el tiempo t :

$$T(t) = T_a + (T_0 - T_a)e^{-kt}$$

En este caso se tienen los valores $T(t) = 34^\circ\text{C}$, $T_a = 31^\circ\text{C}$ y $T_0 = 37^\circ\text{C}$, usando el valor de k_c obtenido anteriormente se establece la siguiente igualdad:

$$34 = 31 + (37 - 31)e^{-0,0913t}$$

$$\frac{34 - 31}{37 - 31} = e^{-0,0913t}$$

$$\ln \frac{1}{2} = \ln e^{-0,0913t}$$

$$\frac{\ln \frac{1}{2}}{-0,0913} = t$$

$$\rightarrow t \approx 7,5920$$

Entonces, la persona falleció hace aproximadamente 7,5920 horas.

11. Ejercicio 11

Enunciado: En clase se explicó la idea básica detrás del método de eliminación Gaussiana con pivoteo total, para la matriz A de tamaño $m \times m$. Recuerde que en este, la idea es intercambiar dos filas y dos columnas, en cada paso k , para ubicar el mayor elemento, es decir:

$$\max_{k \leq i, j \leq m} |A_{i,j}|$$

llamado pivote, en la entrada de la diagonal, en cada paso. Esto genera una factorización $PAQ = LU$, donde L y U son triangular inferior y superior, respectivamente; P contiene todos los cambios de fila y Q los cambios de columna.

(a) Explique con detalle como se construye todo el proceso de eliminación gaussiana con pivoteo total. Escriba explícitamente las fórmulas de las matrices: P , Q , L y U , para el caso general.

En la eliminación gaussiana con pivoteo total, en cada paso k ($k = 1, 2, \dots, m - 1$) se busca el elemento mayor en toda la porción restante de la matriz para minimizar los errores de redondeo:

$$|A_{r_k, c_k}^{(k)}| = \max_{k \leq i, j \leq m} |A_{i,j}^{(k)}|$$

Para colocar dicho pivote en la posición diagonal deseada (k, k) , se realizan simultáneamente intercambios de filas y de columnas. Las fórmulas y definiciones generales de las matrices resultantes son:

- **Matriz de cambio de Filas (P):** Acumula todas las operaciones elementales relacionadas al cambio de filas realizadas:

$$P = P_{m-1}P_{m-2} \dots P_2P_1$$

- **Matriz de cambio de Columnas (Q):** Acumula todas las operaciones elementales relacionadas al cambio de columnas realizadas:

$$Q = Q_1 Q_2 \dots Q_{m-2} Q_{m-1}$$

- **Matriz Triangular Superior (U):** Es la matriz escalonada final obtenida tras aplicar el proceso de reducción:

$$U = A^{(m)}$$

- **Matriz Triangular Inferior (L):** Matriz triangular inferior que almacena todas las operaciones elementales de filas y columnas realizadas a A :

$$L = I + \sum_{k=1}^{m-1} \tilde{l}_k e_k^T \quad \text{donde } \tilde{l}_k = P_{m-1} P_{m-2} \dots P_{k+1} l_k$$

La factorización resultante es $PAQ = LU$

(b) Escriba el Algoritmo del método de eliminación Gaussiana con pivoteo total.

- Una matriz cuadrada A de tamaño $m \times m$.
- Inicializar $U = A$, y las matrices identidad $L = I_m$, $P = I_m$, $Q = I_m$.
- **Para** $k = 1$ hasta $m - 1$ **hacer**:
 - **[Pivoteo Total]** Buscar los índices (i, j) en el rango $k \leq i, j \leq m$ tales que $|U_{i,j}|$ sea el valor máximo absoluto de la submatriz activa.
 - **[Intercambio de Filas]** Si $i \neq k$:
 - Intercambiar la fila k y la fila i en U .
 - Intercambiar la fila k y la fila i en P .
 - Si $k > 1$, intercambiar la fila k y la fila i en las primeras $k - 1$ columnas de L .
 - **[Intercambio de Columnas]** Si $j \neq k$:
 - Intercambiar la columna k y la columna j en U .
 - Intercambiar la columna k y la columna j en Q .
 - **[Proceso de Eliminación]** **Para** $i = k + 1$ hasta m **hacer**:
 - Calcular el multiplicador: $L_{i,k} = \frac{U_{i,k}}{U_{k,k}}$
 - Actualizar la fila i de U : $U_{i,k:m} = U_{i,k:m} - L_{i,k} U_{k,k:m}$
 - **end**
- **end**
- Devolver las matrices P , Q , L y U tales que $PAQ = LU$.

(c) Escriba una función de MATLAB con el método de eliminación Gaussiana con pivoteo total.

Listing 5: Función para el Método de Eliminación Gaussiana con Pivoteo Total

```
1 function [L, U, P, Q] = eliminacionGaussianaPivoteoTotal(A)
2     % ELIMINACIONGAUSSIANAPIVOTEOTOTAL Calcula la factorizacion PAQ =
   LU usando pivoteo total
3
4     m = size(A, 1);
5     U = A;
6     L = eye(m);
7     P = eye(m);
8     Q = eye(m);
9
10    for k = 1 : m-1
11
12        subMatriz = abs(U(k:m, k:m));
13        [maxVal, idxRow] = max(subMatriz, [], 1);
14        [~, s_rel] = max(maxVal);
15        r_rel = idxRow(s_rel);
16
17        r = r_rel + k - 1;
18        s = s_rel + k - 1;
19
20        if r ~= k
21            U([k, r], k:m) = U([r, k], k:m);
22            P([k, r], :) = P([r, k], :);
23            if k > 1
24                L([k, r], 1:k-1) = L([r, k], 1:k-1);
25            end
26        end
27
28        if s ~= k
29            U(:, [k, s]) = U(:, [s, k]);
30            Q(:, [k, s]) = Q(:, [s, k]);
31        end
32
33        for i = k+1 : m
34            if U(k, k) == 0
35                error('El pivote es cero. La matriz puede que sea
   singular.');
```

12. Ejercicio 12

Enunciado: Newton para sistemas no lineales

(a) Pruebe que la convergencia del método de Newton para sistemas no lineales es cuadrática.

Considere un sistema de ecuaciones no lineales

$$F(x) = 0, \quad F : \mathbb{R}^n \rightarrow \mathbb{R}^n.$$

Sea $x^* \in \mathbb{R}^n$ una solución del sistema, es decir,

$$F(x^*) = 0,$$

y la matriz Jacobiana de F en x^* ,

$$J(x^*) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \cdots & \frac{\partial F_n}{\partial x_n} \end{pmatrix},$$

es no singular.

Suponga que F es de clase C^2 en un entorno de x^* . Aplicando el teorema de Taylor a F alrededor de x_k , evaluando en x^* ,

$$F(x^*) = F(x_k) + J(x_k)(x^* - x_k) + R_k,$$

donde R_k es el término de resto de Taylor.

Como

$$x^* - x_k = -e_k,$$

$$\implies F(x^*) = F(x_k) - J(x_k)e_k + R_k.$$

Como x^* es solución del sistema,

$$F(x^*) = 0.$$

Entonces,

$$0 = F(x_k) - J(x_k)e_k + R_k,$$

$$\implies F(x_k) = J(x_k)e_k - R_k.$$

Defina $\tilde{R}_k = -R_k$

$$\implies F(x_k) = J(x_k)e_k + \tilde{R}_k.$$

Como $F \in C^2$, el término de resto satisface

$$\|\tilde{R}_k\| \leq L_1 \|e_k\|^2,$$

para alguna constante $L_1 > 0$, siempre que x_k se encuentre suficientemente cerca de x^* . Se tiene que,

$$x_{k+1} = x_k - J(x_k)^{-1}F(x_k).$$

Restando x^* a ambos lados,

$$x_{k+1} - x^* = x_k - x^* - J(x_k)^{-1}F(x_k).$$

$$e_{k+1} = e_k - J(x_k)^{-1}F(x_k).$$

Sustituyendo en la expansión de Taylor,

$$\begin{aligned} e_{k+1} &= e_k - J(x_k)^{-1} \left[J(x_k)e_k + \tilde{R}_k \right] \\ &= e_k - J(x_k)^{-1}J(x_k)e_k - J(x_k)^{-1}\tilde{R}_k. \end{aligned}$$

$$\implies e_{k+1} = e_k - e_k - J(x_k)^{-1}\tilde{R}_k.$$

$$e_{k+1} = -J(x_k)^{-1}\tilde{R}_k.$$

Tomando normas,

$$\|e_{k+1}\| \leq \|J(x_k)^{-1}\| \|\tilde{R}_k\|.$$

Como $J(x^*)$ es no singular y J es continua, existe un entorno de x^* en el cual $J(x)$ permanece no singular y su inversa está acotada. Por tanto, existe una constante $M > 0$ tal que

$$\|J(x_k)^{-1}\| \leq M.$$

Luego,

$$\|\tilde{R}_k\| \leq L_1 \|e_k\|^2.$$

Combinando ambas desigualdades,

$$\|e_{k+1}\| \leq ML_1 \|e_k\|^2.$$

Defina

$$L = ML_1,$$

$$\implies \|e_{k+1}\| \leq L\|e_k\|^2.$$

Por definición, esta desigualdad demuestra que el método de Newton converge localmente con orden cuadrático hacia x^* .

(b) Escriba el sistema como un problema de encontrar su solución

$$\begin{cases} 3x_1 - \cos(x_2x_3) - 0,5 = 0 \\ x_1^2 - 81(x_2 + 0,1)^2 + \sin(x_3) + 1,06 = 0 \\ e^{-x_1x_2} + 20x_3 - \frac{3-10\pi}{3} = 0 \end{cases}$$

(c) Matriz Jacobiana e iteración de Newton

$$J(\mathbf{x}) = \begin{bmatrix} 3 & x_3 \sin(x_2x_3) & x_2 \sin(x_2x_3) \\ 2x_1 & -162(x_2 + 0,1) & \cos(x_3) \\ -x_2 e^{-x_1x_2} & -x_1 e^{-x_1x_2} & 20 \end{bmatrix}$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \begin{bmatrix} 3 & x_3 \sin(x_2x_3) & x_2 \sin(x_2x_3) \\ 2x_1 & -162(x_2 + 0,1) & \cos(x_3) \\ -x_2 e^{-x_1x_2} & -x_1 e^{-x_1x_2} & 20 \end{bmatrix}^{-1} \cdot \begin{bmatrix} 3x_1 - \cos(x_2x_3) - 0,5 \\ x_1^2 - 81(x_2 + 0,1)^2 + \sin(x_3) + 1,06 \\ e^{-x_1x_2} + 20x_3 - \frac{3-10\pi}{3} \end{bmatrix}$$

(d) Iteración del método

$$\mathbf{x}_1 = \mathbf{x}_0 - [J(\mathbf{x}_0)]^{-1} \cdot \mathbf{F}(\mathbf{x}_0)$$

Siguiendo la sugerencia

$$\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{y}_0$$

Para $\mathbf{y}_0$

$$\begin{aligned} J(\mathbf{x}_0) \cdot \mathbf{y}_0 &= -\mathbf{F}(\mathbf{x}_0) \\ \begin{bmatrix} 3 & 9,99983334 \times 10^{-4} & -9,99983334 \times 10^{-4} \\ 0,2 & -32,4 & 0,9950041653 \\ -0,09900498337 & -0,09900498337 & 20 \end{bmatrix} \begin{bmatrix} y_{01} \\ y_{02} \\ y_{03} \end{bmatrix} \\ &= \begin{bmatrix} -1,199995 \\ -2,269833417 \\ 8,462025346 \end{bmatrix} \end{aligned}$$

Resolviendo para $\mathbf{y}_0$

$$\mathbf{y}_0 = \begin{bmatrix} -0,4001657001 \\ 0,08053691275 \\ 0,42152466368 \end{bmatrix}$$

$$\Rightarrow \mathbf{x}_1 = \begin{bmatrix} -0,3001657001 \\ 0,18053691275 \\ 0,32152466368 \end{bmatrix}$$

Así

$$\mathbf{x}_2 = \begin{bmatrix} 1,10348735652 \\ 0,04443825830 \\ -0,51657057441 \end{bmatrix}$$

$$\mathbf{x}_3 = \begin{bmatrix} 0,5001118291 \\ -0,00878303021 \\ -0,52527884270 \end{bmatrix}$$

(e) Algoritmo para la iteración

Entradas: F vector con las funciones del sistema, J la matriz jacobiana, x_0 el vector inicial para aproximar, tol la tolerancia para detener el proceso.

Salidas: la aproximación a para la solución del sistema.

Pasos:

contador = 1

Mientras la tolerancia > error ó iterMax > contador

- evaluar J y F en $\mathbf{x}_k$

- Resolver el sistema $J(\mathbf{x}_k)\mathbf{y}_k = -F(\mathbf{x}_k)$ para $\mathbf{y}_k$

- $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{y}_k$

- error = $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_\infty$

- contador + 1

Fin

(f) código MATLAB para el método

```

1 function x2 = NewtonSistemasNoLineales(F, J, x0, tol)
2 % Devuelve una aproximacion para la solucion de un sistema no lineal
   por
3 % medio del metodo de Newton
4 %
5 % Entradas:   F --- vector con las funciones del sistema
6 %             J --- la matriz jacobiana
7 %             x0 --- aproximacion inicial
8 %             tol --- tolerancia para el criterio de parada
9 %
10 % Salidas:    x --- aproximacion a la solucion
11 %
12
13 x1 = x0(:);

```

```

14 error = inf;
15 contador = 1;
16 while error > tol && contador < 1000
17     Fx = F(x1);
18     Jx = J(x1);
19     y = -Jx \ Fx;
20     x2 = x1 + y;
21     error = norm(x2 - x1, inf);
22     x1 = x2;
23     contador = contador + 1;
24 end
25 end

```

13. Ejercicio 13

Enunciado: Ejercicios sobre estabilidad

(a) Sean $x, y \in \mathbb{C}^m$. Demuestre que calcular el producto interno $x^T y$ es estable hacia atrás

Tenemos que para $i \in \{1, 2, \dots, m\}$ el producto interno $x^T y = \sum_{i=1}^m x_i y_i$, por el primer axioma de F:

$$\begin{aligned} \text{fl}(x_i) &= x_i(1 + \varepsilon_{x,i}) \quad \text{con } |\varepsilon_{x,i}| \leq \varepsilon_M \\ \text{fl}(y_i) &= y_i(1 + \varepsilon_{y,i}) \quad \text{con } |\varepsilon_{y,i}| \leq \varepsilon_M \end{aligned}$$

Ademas por el axioma fundamental de la aritmética en punto flotante:

$$\text{fl}(x^T y) = \sum_{i=1}^m x_i y_i (1 + \varepsilon) \quad \text{con } |\varepsilon| \leq \varepsilon_M$$

Por lo tanto tenemos que sustituyendo:

$$\text{fl}(x^T y) = \sum_{i=1}^m x_i (1 + \varepsilon_{x,i}) y_i (1 + \varepsilon_{y,i}) (1 + \varepsilon)$$

Definimos:

$$\begin{aligned} \tilde{x}_i &= x_i (1 + \varepsilon_{x,i}) (1 + \varepsilon) \\ \tilde{y}_i &= y_i (1 + \varepsilon_{y,i}) \end{aligned}$$

Al sustituir esto en el producto interno:

$$\text{fl}(x^T y) = \sum_{i=1}^m \tilde{x}_i \tilde{y}_i = \tilde{x}^T \tilde{y} = f(\tilde{x}, \tilde{y})$$

Finalmente tenemos que:

$$\frac{|\tilde{x}_i - x_i|}{|x_i|} = |(1 + \varepsilon_{x,i})(1 + \varepsilon_i) - 1| = |\varepsilon_{x,i} + \varepsilon + \varepsilon_{x,i}\varepsilon|$$

Sabemos que $|\varepsilon| \leq \varepsilon_M$ y además $|\varepsilon_{x,i}| \leq \varepsilon_M$. Por desigualdad triangular tenemos que:

$$\frac{|\tilde{x}_i - x_i|}{|x_i|} \leq |\varepsilon_{x,i}| + |\varepsilon| + |\varepsilon_{x,i}\varepsilon| \leq 2\varepsilon_M + \varepsilon_M^2$$

Para un ε_M suficientemente pequeño $\varepsilon_M \leq \varepsilon_M^2$ por lo que:

$$\frac{|\tilde{x}_i - x_i|}{|x_i|} = O(\varepsilon_M)$$

De manera análoga:

$$\frac{|\tilde{y} - y|}{|y|} = O(\varepsilon_M)$$

Por lo tanto, el algoritmo es estable hacia atrás.

b) Efectos del factor de crecimiento

b.1) Escriba una función que reciba como entrada $m \in \mathbb{N}$ y devuelva la matriz $A \in \mathbb{R}^{m \times m}$ dada por

$$A = \begin{bmatrix} 1 & & & & 1 \\ -1 & 1 & & & 1 \\ -1 & -1 & 1 & & 1 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ -1 & -1 & \dots & -1 & 1 & 1 \\ -1 & -1 & \dots & -1 & -1 & 1 \end{bmatrix},$$

donde las entradas de la diagonal y la última columna son 1, y las entradas debajo de la diagonal son -1 .

Listing 6: Función para generar la matriz A

```

1 function A = generarMatrizA(m)
2     % generarMatrizA: Genera la matriz m x m de la Pregunta 13(b)
3     % Entrada:
4     %   m - Dimensión de la matriz cuadrada
5     % Salida:
6     %   A - Matriz m x m con 1s en la diagonal y última columna y -1s
7         debajo de la diagonal.
8
9     A = -tril(ones(m), -1);
10    A = A + eye(m);
11    A(:, end) = 1;
12 end

```

b.2) Para $m = 50$, calcule el factor de crecimiento de A y la norma $\|A - LU\|_2$. Comente sobre los resultados.

$$\rho(A) = \frac{\max |u_{ij}|}{\max |a_{ij}|},$$

y la norma $\|A - LU\|_2$. Comente sobre los resultados.

El factor de crecimiento es $\rho(A) = 2^{m-1} = 2^{49} \approx 5,629 \times 10^{14}$ para $m = 50$. Este numero tan alto indica que los errores de redondeo se propagan y amplifican al momento de resolver el sistema.

b.3)

Para cada $m = 10 : 60$, defina $x_{\text{Exact}} = \text{rand}(m, 1)$ y calcule

$$b = A * x_{\text{Exact}}.$$

Resuelva el sistema $A * x_{\text{Apr}} = b$ mediante la factorización LU y la factorización QR (utilice los comandos `lu` y `qr` de MATLAB). Obtenga una gráfica de $\|x_{\text{Exact}} - x_{\text{Apr}}\|_2$ en función de m . Justifique el comportamiento del gráfico.

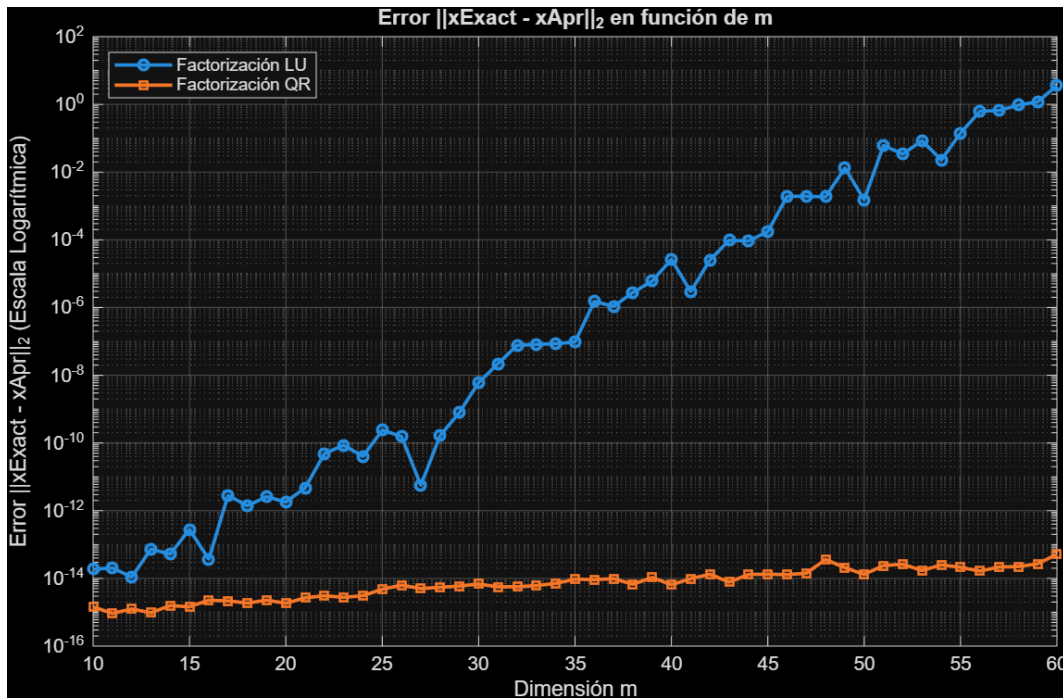


Figura 5: Error $\|x_{\text{Exact}} - x_{\text{Apr}}\|_2$ en escala logarítmica para las factorizaciones LU y QR.

El error al utilizar la factorización **LU** crece exponencialmente con m debido a que la eliminación gaussiana es inestable hacia atrás para esta matriz en particular causa de su elevado factor de crecimiento. En contraste, la factorización **QR** garantiza estabilidad numérica y manteniendo el error en el orden de la precisión de máquina ($\approx 10^{-14}$ a 10^{-15}).

14. Ejercicio 14

Enunciado: Sea $A \in \mathbb{R}^m$. La factorización de Cholesky de una matriz positiva definida A está dada por: $A = LL^t$, donde $L \in \mathbb{R}^m$ es una matriz triangular inferior y L^t su transpuesta que será triangular superior. Suponga que la matriz L tiene la forma:

$$L = \begin{bmatrix} L_{1,1} & 0 & \cdots & 0 \\ L_{2,1} & L_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ L_{n,1} & L_{n,2} & \cdots & L_{n,n} \end{bmatrix}$$

(a) Pruebe que las entradas de la matriz L están dadas por las fórmulas:

$$L_{j,j} = \sqrt{A_{j,j} - \sum_{k=1}^{j-1} (L_{j,k})^2}, \quad \text{y} \quad L_{i,j} = \frac{1}{L_{j,j}} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} \right), \quad \text{para } i > j$$

Solución:

Como indica el enunciado $A = LL^t$, a partir de esta igualdad considere la definición de multiplicación de matrices y el hecho de que A es el producto de una matriz y su transpuesta y verá que las entradas de A vienen dadas por:

$$A_{i,j} = \sum_{k=1}^{\min[i,j]} L_{i,k} L_{j,k}$$

Con esta expresión para las entradas de A se pueden observar los dos casos que se presentan en el enunciado:

Caso 1. $i = j$: Las entradas $A_{j,j}$ se determinan de la siguiente manera:

$$A_{j,j} = \sum_{k=1}^j L_{j,k} L_{j,k} = \sum_{k=1}^j L_{j,k}^2$$

La sumatoria se puede reescribir como:

$$A_{j,j} = \sum_{k=1}^{j-1} (L_{j,k})^2 + (L_{j,j})^2$$

Y entonces:

$$\begin{aligned} A_{j,j} - \sum_{k=1}^{j-1} (L_{j,k})^2 &= (L_{j,j})^2 \\ \rightarrow L_{j,j} &= \sqrt{A_{j,j} - \sum_{k=1}^{j-1} (L_{j,k})^2} \end{aligned}$$

Caso 2. $i > j$: Las entradas $A_{i,j}$ se determinan de la siguiente manera:

$$A_{i,j} = \sum_{k=1}^j L_{i,k} L_{j,k}$$

Nuevamente, se puede reescribir la sumatoria como:

$$A_{i,j} = \sum_{k=1}^{j-1} L_{i,k} L_{j,k} + L_{i,j} L_{j,j}$$

Y entonces:

$$A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} = L_{i,j} L_{j,j}$$

$$A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} = L_{i,j} L_{j,j}$$

$$\rightarrow L_{i,j} = \frac{1}{L_{j,j}} \left(A_{i,j} - \sum_{k=1}^{j-1} L_{i,k} L_{j,k} \right)$$

(b) Usando (a), encuentre la factorización de Cholesky de la matriz A, dada por:

$$\begin{pmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{pmatrix}$$

Solución:

Por el criterio de Sylvester, se sabe que una matriz es definida positiva si es simétrica y si sus menores principales son mayores de cero. En el caso de la matriz A note que:

$$A^t = \begin{pmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{pmatrix} = A$$

Por lo tanto, A es simétrica, se pueden calcular los menores principales de la siguiente manera:

$$\Delta_1 = A_{1,1} = 4 > 0$$

$$\Delta_2 = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix} = \begin{vmatrix} 4 & 12 \\ 12 & 37 \end{vmatrix} = 4 > 0$$

$$\Delta_3 = \det(A) = 36 > 0$$

Como todos los menores principales son mayores a cero entonces la matriz A si es definida positiva.

Se pueden aplicar las formulas del inciso anterior para calcular las entradas de la matriz L y así construir la factorización de Cholesky de A .

Para la primera columna:

$$L_{1,1} = \sqrt{A_{1,1}} = 2 \quad L_{2,1} = \frac{A_{2,1}}{L_{1,1}} = 6 \quad L_{3,1} = \frac{A_{3,1}}{L_{1,1}} = -8$$

Para la segunda columna:

$$L_{2,2} = \sqrt{A_{2,2} - (L_{2,1})^2} = 1 \quad L_{2,3} = \frac{1}{L_{2,2}}(A_{3,2} - L_{3,1}L_{2,1}) = 5$$

Para la tercera columna:

$$L_{3,3} = \sqrt{A_{3,3} - (L_{3,1})^2 - (L_{3,2})^2} = 3$$

Note que para la segunda y tercer columna no es necesario calcular algunas entradas pues L es una matriz triangular inferior, por lo que las entradas sobre la diagonal serán igual a cero.

Entonces,

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{pmatrix}$$

$$L^t = \begin{pmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{pmatrix}$$

Por lo tanto:

$$A = LL^t$$

15. Ejercicio 15

Enunciado: Ejercicio de investigación: método de Muller

(a) Explique detalladamente el método de Muller para la resolución numérica de ecuaciones no lineales

Como una alternativa a los métodos de Secante, Newton-Raphson y demás que se han estudiado, el método de Muller funciona para aproximar las soluciones o ceros de un polinomio de grado n .

De la manera similar a como el método de Secante aproxima los ceros de un polinomio por medio de una recta que pasa por dos puntos, Muller intenta hacer lo mismo pero con una parábola que pasa por tres puntos dados.

Considere como punto de partida el siguiente polinomio cuadrático que contiene en su gráfico a los puntos $(p_0, f(p_0))$, $(p_1, f(p_1))$ y $(p_2, f(p_2))$:

$$P(x) = a(x - p_2)^2 + b(x - p_2) + c$$

Como pasa por los tres puntos establecidos anteriormente se puede decir que:

$$f(p_0) = a(p_0 - p_2)^2 + b(p_0 - p_2) + c$$

$$f(p_1) = a(p_1 - p_2)^2 + b(p_1 - p_2) + c$$

$$f(p_2) = a(p_2 - p_2)^2 + b(p_2 - p_2) + c$$

De las ecuaciones anteriores se pueden obtener expresiones para a , b y c en términos de los puntos de coordenadas establecidos.

$$c = f(p_2),$$

$$b = \frac{(p_0 - p_2)^2 [f(p_1) - f(p_2)] - (p_1 - p_2)^2 [f(p_0) - f(p_2)]}{(p_0 - p_2)(p_1 - p_2)(p_0 - p_1)},$$

$$a = \frac{(p_1 - p_2) [f(p_0) - f(p_2)] - (p_0 - p_2) [f(p_1) - f(p_2)]}{(p_0 - p_2)(p_1 - p_2)(p_0 - p_1)}.$$

Para la siguiente aproximación, el método de Muller propone que el siguiente valor p_3 se debe calcular con la siguiente fórmula:

$$p_3 = p_2 - \frac{2c}{b \pm \sqrt{b^2 - 4ac}}$$

Debido a la magnitud del denominador, se dice que p_3 es la aproximación a un cero de $P(x)$ más cercana a p_2 si esta aproximación no es satisfactoria, se puede iterar nuevamente el método, esta vez se usarían las aproximaciones iniciales p_1 , p_2 y p_3 , y así sucesivamente hasta obtener una aproximación aceptable.

Se debe destacar que este método es ideal para los casos donde existe la posibilidad de que los ceros del polinomio sean números complejos, incluso si todos sus coeficientes son números reales.

El algoritmo de este método se puede definir de la siguiente manera:

Entradas:

- p_0 , p_1 y p_2 , los tres puntos por donde pasa la parábola.
- Tolerancia de convergencia
- N , cantidad máxima de iteraciones

Salida:

- p , el cero aproximado para el polinomio.

Paso 1: Defina:

$$\begin{aligned}h_1 &= p_1 - p_0, \\h_1 &= p_2 - p_1, \\ \delta_1 &= \frac{f(p_1) - f(p_0)}{h_1}, \\ \delta_2 &= \frac{f(p_2) - f(p_1)}{h_2}, \\ d &= \frac{\delta_1 - \delta_2}{h_2 + h_1}, \\ i &= 3\end{aligned}$$

Paso 2: Mientras $i \leq N$ siga los pasos del 3 al 7

Paso 3: Defina:

$$\begin{aligned}b &= \delta_2 + h_2 d, \\ D &= \sqrt{b^2 - 4f(p_2)d}\end{aligned}$$

Paso 4: Si $|b - D| < |b + D|$ entonces defina:

$$E = b + D$$

De lo contrario:

$$E = b - D$$

Paso 5: Defina:

$$\begin{aligned}h &= \frac{-2f(p_2)}{E}, \\ p &= p_2 + h\end{aligned}$$

Paso 6: Si $|h| < \text{tolerancia}$ entonces devuelva el valor de p , la aproximación calculada y detenga el método

Paso 7: Para preparar la siguiente iteración defina:

$$\begin{aligned}
p_0 &= p_1, \\
p_1 &= p_2, \\
p_2 &= p, \\
h_1 &= p_1 - p_0, \\
h_1 &= p_2 - p_1, \\
\delta_1 &= \frac{f(p_1) - f(p_0)}{h_1}, \\
\delta_2 &= \frac{f(p_2) - f(p_1)}{h_2}, \\
d &= \frac{\delta_1 - \delta_2}{h_2 + h_1}, \\
i &= i + 1
\end{aligned}$$

Paso 8: En caso de sobrepasar la cantidad máxima de iteraciones, devolver un mensaje de error indicando que el método no fue exitoso:

(b) Realice una iteración del método de Muller para el siguiente polinomio

$$f(x) = 16x^4 - 40x^3 + 5x^2 + 20x + 6$$

Solución:

Primero, se deben evaluar los valores de $f(x)$ en los puntos que se brindan en el enunciado ($x_0 = \frac{1}{2}$, $x_1 = -\frac{1}{2}$ y $x_2 = 0$)

$$\begin{aligned}
f(x_0) &= f\left(\frac{1}{2}\right) = 16 \cdot \frac{1}{16} - 40 \cdot \frac{1}{8} + 5 \cdot \frac{1}{4} + 20 \cdot \frac{1}{2} + 6 = \frac{53}{4}, \\
f(x_1) &= f\left(-\frac{1}{2}\right) = 16 \cdot \frac{1}{16} + 40 \cdot \frac{1}{8} + 5 \cdot \frac{1}{4} - 20 \cdot \frac{1}{2} + 6 = \frac{13}{4}, \\
f(x_2) &= f(0) = 6.
\end{aligned}$$

Ahora, se definen las cantidades necesarias para el método establecidas en el paso 1 del algoritmo:

$$\begin{aligned}
h_1 &= x_1 - x_0 = -\frac{1}{2} - \frac{1}{2} = -1, \\
h_2 &= x_2 - x_1 = 0 - \left(-\frac{1}{2}\right) = \frac{1}{2}, \\
\delta_1 &= \frac{f(x_1) - f(x_0)}{h_1} = \frac{\frac{13}{4} - \frac{53}{4}}{-1} = 10, \\
\delta_2 &= \frac{f(x_2) - f(x_1)}{h_2} = \frac{6 - \frac{13}{4}}{\frac{1}{2}} = \frac{11}{2}, \\
d &= \frac{\delta_2 - \delta_1}{h_2 + h_1} = \frac{\frac{11}{2} - 10}{\frac{1}{2} - 1} = \frac{-\frac{9}{2}}{-\frac{1}{2}} = 9.
\end{aligned}$$

Seguidamente, se determina la constante b de la parábola que pasa por los puntos iniciales y el discriminante de esta.

$$\begin{aligned}
b &= \delta_2 + h_2 d = \frac{11}{2} + \frac{1}{2} \cdot 9 = 10 \\
D &= \sqrt{b^2 - 4f(x_2)d} = \sqrt{100 - 4(6)(9)} = \sqrt{-116} = 2\sqrt{29}i
\end{aligned}$$

Con el valor del discriminante y b se puede determinar E , note que:

$$\begin{aligned}
|b - D| &= \left| 10 - 2\sqrt{29}i \right| \approx 14,697, \\
|b + D| &= \left| 10 + 2\sqrt{29}i \right| \approx 14,697.
\end{aligned}$$

Como ambos son iguales se elije.

$$E = b - D = 10 - 2\sqrt{29}i$$

Se procede a calcular h y realizar la aproximación x .

$$\begin{aligned}
h &= \frac{-2f(x_2)}{E} = \frac{-12}{10 - 2\sqrt{29}i} = -\frac{5}{9} - \frac{\sqrt{29}}{9}i \\
x &= x_2 + h = -\frac{5}{9} - \frac{\sqrt{29}}{9}i \approx -0,5556 - 0,5984i
\end{aligned}$$

Por último, se calcula el error absoluto de esta iteración.

$$|x_3 - x_2| = \sqrt{\frac{25}{81} + \frac{29}{81}} = \sqrt{\frac{54}{81}} \approx 0,8165.$$

c) Escriba una función en MATLAB con el método de Müller descrito en la parte (a)

La implementación del método de Muller en Matlab es la siguiente:

Listing 7: Función en MATLAB para el Método de Müller

```
1 function [M, p_sig] = muller_method(p, p0, p1, p2, tol, N)
2 % MULLER_METHOD Implementa el algoritmo del metodo de Muller .
3 %
4 % Entradas:
5 % p - Vector con los coeficientes del polinomio [a_n, ..., a_1,
6 % a_0]
7 % p0, p1, p2 - Tres puntos iniciales dados
8 % tol - Tolerancia
9 % N - Cantidad maxima de iteraciones
10 %
11 % Salidas:
12 % M - Matriz de resultados
13 % p_sig - Ultima aproximacion obtenida
14
15 h1 = p1 - p0;
16 h2 = p2 - p1;
17
18 f0 = polyval(p, p0);
19 f1 = polyval(p, p1);
20 f2 = polyval(p, p2);
21
22 delta1 = (f1 - f0) / h1;
23 delta2 = (f2 - f1) / h2;
24 d = (delta2 - delta1) / (h2 + h1);
25
26 M = [];
27 i = 3;
28 while i <= N
29     b = delta2 + h2 * d;
30     D = sqrt(b^2 - 4 * f2 * d);
31
32     if abs(b - D) < abs(b + D)
33         E = b + D;
34     else
35         E = b - D;
36     end
37
38     h = -2 * f2 / E;
39     p_sig = p2 + h;
40
41     error_abs = abs(p_sig - p2);
```

```

42     M(end+1, :) = [p_sig, error_abs];
43
44     if error_abs < tol
45         break;
46     end
47
48     p0 = p1;
49     p1 = p2;
50     p2 = p_sig;
51
52     f0 = polyval(p, p0);
53     f1 = polyval(p, p1);
54     f2 = polyval(p, p2);
55
56     h1 = p1 - p0;
57     h2 = p2 - p1;
58     delta1 = (f1 - f0) / h1;
59     delta2 = (f2 - f1) / h2;
60     d = (delta2 - delta1) / (h2 + h1);
61
62     i = i + 1;
63 end
64 end

```

(d) Cálculo de la aproximación x_8 y tabla de resultados

Usando la función programada se calculan las aproximaciones x_8

Cuadro 4: Resultados de las iteraciones del Método de Müller

Iteración (k)	Aproximación (x_k)	Error Absoluto ($ x_k - x_{k-1} $)
1	$-0,555556 - 0,598352i$	0,816497
2	$-0,435450 - 0,102101i$	0,510578
3	$-0,390631 - 0,141852i$	0,059907
4	$-0,357698 - 0,169926i$	0,043275

Resultado Final: La aproximación final obtenida tras las iteraciones correspondientes es:

$$x_8 \approx -0,357698 - 0,169926i$$